

Université Cergy Pontoise

Master 2 IISC- Projet de synthèse



Auto Fleet

Auteurs : Ould Younes Massyl ; Oulfid Karim ; Jianke Lin ; Kotokpo Josué

Rapporteur : Gaussier Philippe

Tuteurs techniques : Abdelwahed Medhi / D'Urso Raphaël

Encadrant gestion de projet : Tianxiao Liu

08/06/2026

Remerciements

Cette section sera consolidée dans la version finale du manuscrit.

Table des matières

Remerciements	1
Table des matières	2
Table des figures	6
Liste des tableaux	7
1 Introduction	8
1.1 Contexte du projet	8
1.2 Un exemple d'utilisation : mise en scénario	9
1.3 Objectifs du projet et fonctionnalités attendues	9
Structure du rapport	11
2 Conception et vue globale du projet	13
2.1 Architecture technique globale	13
2.1.1 Vue d'ensemble du système	13
2.1.2 Sous-systèmes embarqués des robots	13
2.1.3 Communication et supervision de la flotte	14
2.2 Environnement de développement du projet	15
2.2.1 Outils logiciels utilisés	15
2.2.2 Environnement matériel et simulation	15
2.2.3 Outils de collaboration et de gestion de version	16
3 Architecture embarquée des robots autonomes	17
3.1 Contraintes et besoins de l'architecture embarquée	17
3.1.1 Besoins fonctionnels des robots	17
3.1.2 Contraintes mécaniques et énergétiques	18
3.1.3 Choix d'une architecture embarquée distribuée	19
3.2 Conception matérielle et intégration des robots	20
3.2.1 Structure mécanique et châssis	20
3.2.2 Capteurs, cartes embarquées et actionneurs	21
3.2.3 Intégration des différents sous-systèmes	21

3.3	Validation de l'architecture embarquée	22
3.3.1	Tests de locomotion et de commande moteur	22
3.3.2	Problèmes rencontrés lors de l'intégration	22
3.3.3	Ajustements et améliorations apportés	23
3.4	SLAM collaboratif et cartographie partagée	23
3.4.1	Problématique du SLAM multi-agents	23
3.4.2	Exploration autonome par frontières	23
3.4.3	Fusion des cartes partielles	24
3.4.4	Architecture réseau et flux de données	24
3.4.5	Cohérence globale, limites et validation du SLAM collaboratif	25
4	Perception de l'environnement par capteurs embarqués	27
4.1	Besoins en perception et choix des capteurs	27
4.1.1	Rôle de la perception dans le projet	27
4.1.2	Justification du choix des capteurs	27
4.1.3	Limites et complémentarité des capteurs	28
4.2	Traitement et exploitation des données capteurs	29
4.2.1	Acquisition et filtrage des données	29
4.2.2	Fusion des informations issues des capteurs	29
4.2.3	Cartographie partagée par SLAM	30
4.2.4	Estimation d'état et prise de décision locale	30
4.3	Évaluation des performances de perception	31
4.3.1	Détection d'obstacles et estimation de distance	31
4.3.2	Robustesse des mesures en environnement réel	31
4.3.3	Limites actuelles et pistes d'amélioration	31
5	Communication entre robots et supervision (V2X)	33
5.1	Enjeux de communication et de supervision de la flotte	33
5.1.1	Besoins d'échange entre robots et station	33
5.1.2	Contraintes réseau et priorités fonctionnelles	34
5.1.3	Place de la supervision dans l'architecture globale	35
5.2	Architecture V2X mise en oeuvre	36
5.2.1	Decoupage logiciel de la station	36
5.2.2	Organisation des topics et des messages	36
5.2.3	Agent Raspberry Pi et services persistants	37
5.2.4	Adaptation aux changements de réseau	37
5.3	Supervision Web et pilotage opérateur	38
5.3.1	Vue flotte et état consolidé	38
5.3.2	téléopération et commandes de mission	39

5.3.3	Video wall et profils de qualité	39
5.3.4	Diagnostic réseau et unites de débit	39
5.4	intégration du robot réel R1	40
5.4.1	Chaine de démarrage retenue	40
5.4.2	Video réelle et reduction de latence	41
5.4.3	Preparation de l'extension capteurs	41
5.5	Validation et analyse des limites	42
5.5.1	Resultats obtenus	42
5.5.2	Limites observees	42
5.5.3	améliorations possibles	43
6	Rendu final	44
6.1	Interface utilisateur finale	44
6.1.1	Organisation générale de l'IHM	44
6.1.2	Interactions entre l'utilisateur et le système	44
6.1.3	Informations affichées et retour du système	44
6.2	Tests utilisateur et certification	44
6.2.1	Scénarios de test retenus	45
6.2.2	Résultats observés du point de vue utilisateur	45
6.2.3	Autres tests et limites actuelles	45
7	Gestion de projet	46
7.1	Méthode de gestion	46
7.1.1	Cycle de vie agile utilisé	46
7.1.2	Organisation générale des axes de travail	47
7.1.3	Planning prévisionnel et planning réalisé	48
7.1.4	Versions intermédiaires prévues et réalisées	48
7.2	Répartition des tâches et outils de gestion	49
7.2.1	Répartition initiale des tâches	49
7.2.2	Évolution de la répartition pendant le projet	50
7.2.3	Outils de gestion et de collaboration	51
7.3	Changements rencontrés au cours du projet	52
7.3.1	Problèmes matériels rencontrés	52
7.3.2	Itérations sur les pièces 3D	52
7.3.3	Conséquences sur l'organisation	53
7.4	Réactions de l'équipe et solutions apportées	53
7.4.1	Poursuite du travail en parallèle	53
7.4.2	Utilisation de la simulation	54
7.4.3	Adaptations techniques apportées	54

7.4.4	Bilan de la gestion agile	55
8	Conclusion et perspectives	56
8.1	Conclusion du projet	56
8.2	Perspectives d'amélioration du projet	56
	Bibliographie	57
A	Annexes	58

Table des figures

1.1	Scénario d’exploration collaborative en zone forestière	9
1.2	Diagramme de cas d’utilisation du système Auto Fleet	11
2.1	Architecture embarquée type d’un robot de la flotte Auto Fleet	14
3.1	Flux de données associés à la cartographie partagée	25
4.1	Chaîne de perception et d’estimation d’état du projet Auto Fleet	30
5.1	Position de la supervision dans l’architecture globale	35
5.2	Architecture logicielle complétée de la communication V2X	38
5.3	Organisation générale de l’IHM de supervision	40
5.4	Vue de diagnostic réseau et de gestion des flux video	42

Liste des tableaux

2.1	Principales problématiques techniques du projet Auto Fleet	16
3.1	Estimation de la consommation électrique des robots de classe A et B	19
3.2	Signification des variables de la matrice cinématique des roues mécanum	20
3.3	Exemples de flux ROS 2 utilisés dans l'architecture embarquée	21
3.4	Estimation de la bande passante utilisée par un robot de classe A	25
4.1	Capteurs retenus dans l'architecture de perception d'Auto Fleet	28
5.1	Principales familles d'échanges dans la chaîne V2X	34
5.2	Indicateurs réseau utilisés pour qualifier la supervision	35
5.3	Services exécutés sur la station de supervision	36
5.4	Exemples de topics utilisés dans la chaîne V2X	37
5.5	Profil de qualité disponibles pour la video wall	39
5.6	État d'intégration du robot réel R1	41
5.7	Synthèse des limites et perspectives de la chaîne V2X	43
7.1	Cycle de travail itératif utilisé pendant le projet	47
7.2	Structuration du projet Auto Fleet par axes de travail	47
7.3	Comparaison entre le planning prévisionnel et le planning réalisé	48
7.4	Versions intermédiaires prévues et réalisées pendant le projet	49
7.5	Répartition des tâches entre les membres du groupe	50
7.6	Évolution de l'organisation pendant le projet	51
7.7	Outils utilisés pour la gestion et la collaboration	51
7.8	Principaux changements et problèmes matériels rencontrés	52
7.9	Corrections réalisées sur les pièces imprimées en 3D	53
7.10	Réactions mises en place face aux blocages	54
7.11	Adaptations techniques réalisées pendant le projet	55

Chapitre 1

Introduction

1.1 Contexte du projet

Le développement des **véhicules autonomes** et des **systèmes robotiques coopératifs** ne relève plus d'un simple exercice expérimental. Dans les domaines de la **logistique**, de l'**inspection**, de la **surveillance**, de l'**industrie** ou encore des **environnements intelligents**, la question centrale n'est plus uniquement celle de l'autonomie d'un robot isolé, mais celle de la capacité d'un ensemble d'agents à **percevoir**, **décider**, **se coordonner** et **maintenir un service robuste** malgré les incertitudes du terrain.

L'orientation initiale du projet Auto Fleet portait sur la conception d'un robot autonome unique. Une telle approche permettait de traiter les briques fondamentales de la robotique mobile : **perception locale**, **odométrie**, **commande**, **évitement d'obstacles** et **prise de décision embarquée**. Ce cadre demeurerait pertinent pour poser une base méthodologique solide. Il introduisait néanmoins une limite structurelle : il n'ouvrait que partiellement la voie aux problématiques de **coopération inter-agents**, de **partage de données** et de **supervision distribuée**.

Le projet a donc été réorienté vers une flotte de robots autonomes. Ce changement n'est pas un simple élargissement quantitatif. Il modifie la nature même du système étudié. Dès lors qu'une mission est répartie entre plusieurs unités, apparaissent des exigences nouvelles : **synchronisation**, **communication réseau**, **cohérence des états**, **cartographie partagée**, **gestion des défaillances partielles** et **hétérogénéité des rôles**. La difficulté ne réside plus exclusivement dans la navigation individuelle, mais dans l'intégration d'un ensemble cohérent de sous-systèmes physiques, logiciels et communicationnels.

Dans Auto Fleet, chaque robot doit ainsi maintenir une autonomie minimale de fonctionnement : acquérir des mesures, estimer son état, exécuter une commande locale, transmettre des informations utiles et rester dans un état sûr en cas de dégradation partielle de la chaîne logicielle ou réseau. À cette autonomie locale s'ajoute une logique collective : certaines données doivent être mutualisées, certains statuts partagés, et certaines représentations de l'environnement consolidées à l'échelle de la flotte.

Le projet s'inscrit en outre dans une perspective à la fois **technique**, **expérimentale** et **pédagogique**. Les plateformes industrielles multi-robots déjà disponibles sont souvent coûteuses, peu ouvertes ou difficiles à adapter à un cadre de développement universitaire. Auto Fleet vise au contraire une base de travail **modulaire**, **appropriable** et **évolutive**, sur laquelle peuvent être étudiés de façon concrète la **locomotion omnidirectionnelle**, la **fusion capteurs**, le **SLAM collaboratif**, la **supervision réseau** et la **gestion d'une flotte hétérogène**.

1.2 Un exemple d'utilisation : mise en scénario

Un scénario représentatif du projet consiste en une mission d'exploration d'une zone forestière partiellement inconnue. L'environnement considéré n'est ni parfaitement structuré ni entièrement prédictible : il contient des obstacles naturels, des zones masquées et des passages dont l'accessibilité ne peut être présumée. Dans un tel contexte, le recours à une flotte présente un intérêt opérationnel direct : la surface explorée peut être couverte plus rapidement, les rôles peuvent être répartis, et les informations utiles peuvent être consolidées à mesure de l'avancement de la mission.

La flotte considérée dans ce scénario comprend quatre robots. Deux d'entre eux sont équipés d'un **LiDAR** et participent à la construction progressive d'une carte de l'environnement selon une logique de **SLAM**. Les deux autres s'appuient principalement sur une **caméra** et sur leur perception locale pour progresser dans la zone, compléter l'observation du terrain et contribuer à la recherche de la cible. Cette répartition n'introduit pas une hiérarchie entre robots ; elle formalise plutôt une **spécialisation fonctionnelle** destinée à réduire le coût embarqué tout en conservant une capacité collective élevée.

Pendant la mission, chaque robot exécute une navigation autonome de proximité, avec détection locale d'obstacles, adaptation de trajectoire et remontée d'état. En parallèle, des échanges réseau permettent de diffuser des informations de **position**, de **téléométrie**, d'**événements de mission** et, pour les robots concernés, des données utiles à la **cartographie partagée**. L'opérateur n'assure donc pas un pilotage point à point : il supervise un système dans lequel la décision locale demeure embarquée, tandis que la station centrale conserve une vue consolidée de la situation.

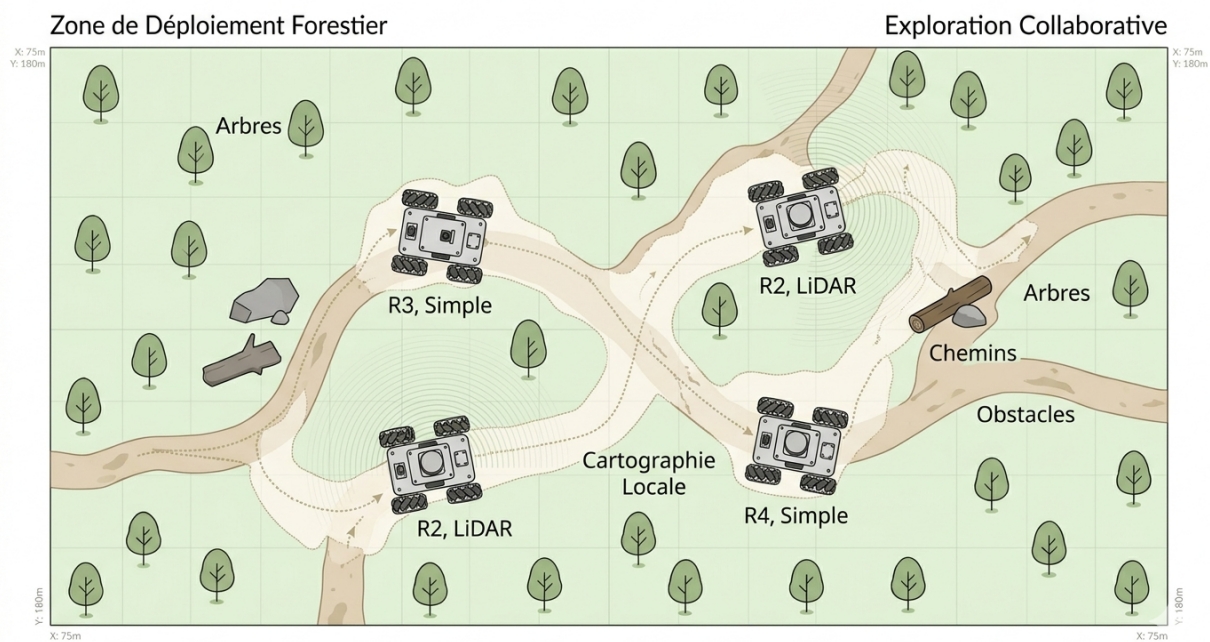


FIGURE 1.1 – Scénario d'exploration collaborative en zone forestière

1.3 Objectifs du projet et fonctionnalités attendues

L'objectif principal du projet Auto Fleet est la conception d'une **flotte de robots autonomes collaboratifs**, depuis l'architecture embarquée jusqu'aux mécanismes de supervision. Il ne s'agit pas uniquement de faire fonctionner plusieurs robots en parallèle, mais de structurer un système capable d'assurer la **cohérence fonctionnelle** d'un ensemble de sous-systèmes répartis.

Un premier objectif concerne l'**architecture matérielle embarquée**. Chaque robot doit intégrer une unité de calcul, un microcontrôleur, des capteurs, des moteurs, une alimentation et des interfaces de communication dans un encombrement raisonnable. Le système ne peut être viable que si cette intégra-

tion reste **maintenable**, **diagnostiquable** et suffisamment **modulaire** pour accepter des itérations de développement.

Un second objectif porte sur la **locomotion** et la **perception**. Les robots doivent être capables de se déplacer, d’asservir leurs roues, d’estimer leur mouvement et de produire une perception exploitable de leur voisinage. Cette perception repose sur plusieurs sources : **LiDAR**, **caméra**, **IMU**, **encodeurs** et, selon les cas, capteurs de proximité. L’intérêt n’est pas seulement d’accumuler des mesures, mais d’en faire une base suffisamment robuste pour soutenir la navigation et la supervision.

Le projet vise également la mise en place d’une **cartographie partagée**. Les robots équipés d’un LiDAR doivent contribuer à la construction d’une représentation de l’environnement qui dépasse leur trajectoire individuelle. Cette carte n’a pas une valeur illustrative ; elle constitue une structure de référence pour la mission, pour la lecture de l’état global et, à terme, pour l’aide à la navigation des unités moins instrumentées.

Enfin, Auto Fleet intègre une dimension marquée de **communication** et de **supervision**. La flotte doit être observable : l’opérateur doit pouvoir identifier les robots disponibles, visualiser leur état, suivre leur progression et intervenir sans se substituer à la logique embarquée. Cette exigence conduit à structurer une chaîne complète allant du robot au backend, puis du backend à l’interface de supervision.

Les fonctionnalités attendues du système sont les suivantes :

- **Déplacement autonome** de robots mobiles en environnement partiellement inconnu, avec maintien d’un comportement sûr ;
- **Commande bas niveau** des moteurs et gestion de la locomotion à quatre roues, y compris pour les plateformes à roues mécanum ;
- **Perception locale** par combinaison de **LiDAR**, **caméra**, **IMU**, **encodeurs** et capteurs de proximité ;
- **Détection d’obstacles** et adaptation locale de trajectoire ;
- **Cartographie partagée** de la zone explorée par les robots instrumentés pour le **SLAM** ;
- **Communication réseau** entre robots, backend et station de supervision ;
- **Simulation** des comportements et de l’interaction avec l’environnement ;
- **Visualisation et contrôle** de la flotte via une **IHM** dédiée.

Auto Fleet a ainsi une double portée. D’une part, il constitue un **démonstrateur technique** permettant d’éprouver une architecture multi-robots cohérente. D’autre part, il fournit une base réutilisable pour des développements ultérieurs en **navigation autonome**, **coordination distribuée**, **cartographie collaborative** et **supervision d’essaims robotiques**.

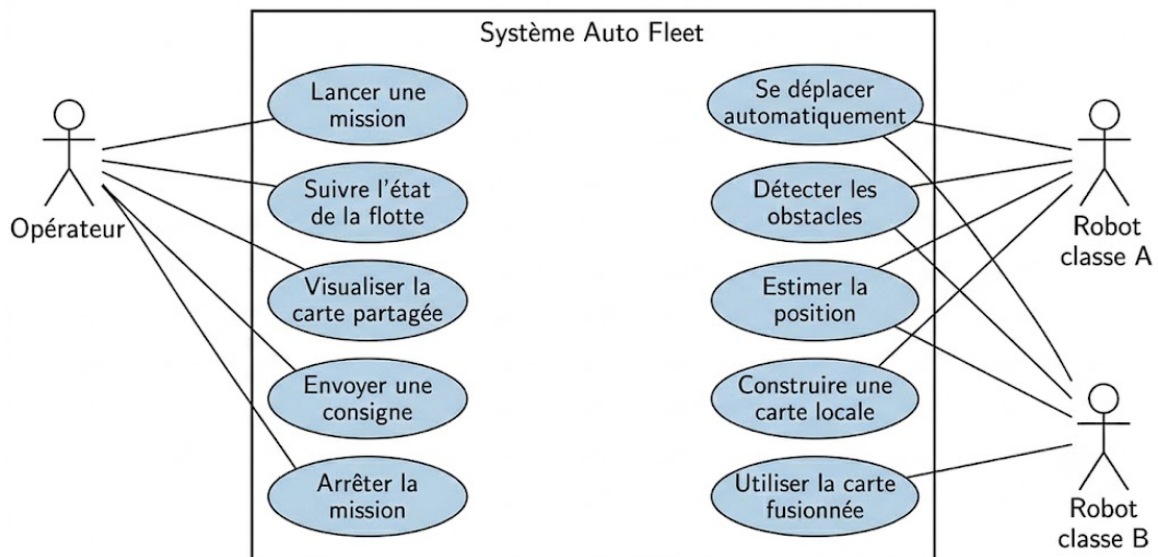


FIGURE 1.2 – Diagramme de cas d'utilisation du système Auto Fleet

Structure du rapport

Afin d'éviter une lecture morcelée, le rapport est organisé selon une progression qui va du **cadre général** vers les **couches techniques**, puis vers les éléments de **validation** et de **gestion de projet**. Cette structuration n'a pas été pensée comme un simple enchaînement rédactionnel ; elle reflète l'ordre logique selon lequel les choix techniques deviennent intelligibles.

- **Chapitre 2 — Conception et vue globale du projet.** Ce chapitre a pour fonction de fixer le **socle architectural** du système. Il présente la logique distribuée retenue, la séparation entre fonctions embarquées et supervision, ainsi que l'environnement de développement qui a permis d'itérer sur le projet. L'enjeu de ce chapitre n'est pas encore d'entrer dans le détail composant par composant, mais de poser la lecture d'ensemble : quels sont les blocs du système, comment circulent les données, quelles briques logicielles ont été mobilisées et selon quel découpage fonctionnel la flotte est structurée. Il sert également de point d'ancrage pour les chapitres suivants, en explicitant les choix de base qui justifient à la fois la conception matérielle, la perception, la communication et la supervision.
- **Chapitre 3 — Architecture embarquée des robots autonomes.** Ce chapitre constitue le noyau matériel du rapport. Il traite des **contraintes fonctionnelles**, des **contraintes mécaniques**, des limites d'**alimentation**, du découpage entre calcul haut niveau et commande temps réel, ainsi que de l'intégration effective des composants sur les plateformes. Une attention particulière y est portée à la notion d'**architecture distribuée embarquée**, indispensable dès lors qu'un système Linux généraliste ne peut suffire à garantir seul les exigences de réactivité des boucles moteur. Le chapitre documente aussi la phase d'intégration, c'est-à-dire le moment où les hypothèses théoriques rencontrent les réalités du câblage, des vibrations, des chutes de tension, des synchronisations temporelles et des compromis d'encombrement. Enfin, il ouvre naturellement sur la question du **SLAM collaboratif**, en montrant que la cartographie ne peut être dissociée de la stabilité de l'architecture physique qui la supporte.
- **Chapitre 4 — Perception de l'environnement par capteurs embarqués.** La perception n'est pas traitée ici comme un module accessoire. Elle constitue la condition d'existence même de l'autonomie. Ce chapitre expose les besoins de captation du projet, justifie le choix des capteurs, discute leurs limites respectives et formalise la manière dont leurs données sont acquises, filtrées, fusionnées et exploitées. Il montre en particulier pourquoi une approche mono-capteur

serait insuffisante dans le cadre d'une flotte évoluant en environnement partiellement inconnu. Le lecteur y trouvera les éléments nécessaires pour comprendre la production de l'odométrie, l'estimation d'état, l'appui du **LiDAR** sur la cartographie, le rôle de l'**IMU** dans la stabilité des estimations, ainsi que la contribution de la **caméra** et des capteurs de proximité à la robustesse globale du système. Ce chapitre prépare également l'analyse critique des performances observées en conditions réelles.

- **Chapitre 5 — Communication entre robots et supervision.** Ce chapitre se concentre sur la **chaîne V2X** au sens large : échanges entre robots, backend, broker de messages et interface opérateur. Il ne se limite pas à décrire une pile de communication ; il interroge la place de la supervision dans un système qui, par principe, doit conserver une part d'autonomie locale. Les différents types de messages y sont distingués selon leur criticité, leur fréquence et leur charge potentielle sur le réseau. La logique de l'**IHM** y est également abordée sous un angle fonctionnel : il s'agit moins de détailler une interface écran par écran que de montrer comment sont articulés **contrôle, visibilité du système, diagnostic réseau, accès vidéo et retours de mission**. Ce chapitre a enfin une dimension évaluative, puisqu'il introduit les métriques de qualité de service utiles pour juger la stabilité réelle de la supervision.
- **Chapitre 6 — Rendu final.** Ce chapitre est volontairement orienté vers la **présentation consolidée du prototype**. Il accueillera les éléments démontrables du système : organisation finale de l'interface, interactions réellement exposées à l'utilisateur, scénarios de test retenus et résultats visibles depuis le point de vue opérateur. Sa fonction n'est pas de répéter les détails d'architecture déjà exposés, mais de rendre compte de ce qui est effectivement stabilisé au moment de la démonstration. Il s'agit donc d'un chapitre de **synthèse opérationnelle**, où devront converger les choix de conception, les campagnes de tests et les contraintes réellement surmontées dans la version présentée.
- **Chapitre 7 — Gestion de projet.** La cohérence d'Auto Fleet ne peut être comprise sans un retour explicite sur l'organisation du travail. Ce chapitre documente la méthode adoptée, la répartition des responsabilités, les ajustements imposés par les retards matériels, le rôle joué par la simulation lorsqu'une partie de l'intégration réelle demeurerait incomplète, ainsi que les outils de coordination mobilisés par l'équipe. Il ne s'agit pas d'un ajout périphérique : dans un projet à forte composante d'intégration, la gestion de projet influence directement les choix techniques, les priorités de développement et le rythme de validation des briques critiques. Ce chapitre permet donc de rendre lisible la manière dont les décisions ont été arbitrées, révisées et, parfois, contraintes par les réalités du calendrier et de l'approvisionnement.
- **Chapitre 8 — Conclusion et perspectives.** Le dernier chapitre clôt le rapport par une synthèse des résultats atteints, des verrous demeurant ouverts et des prolongements techniquement crédibles. Il doit permettre de distinguer ce qui relève d'une **preuve de faisabilité**, ce qui relève d'un **prototype partiellement stabilisé** et ce qui constitue encore une **perspective de développement**. Cette conclusion ne vise pas à produire une clôture rhétorique ; elle doit au contraire laisser apparaître les marges d'amélioration concernant la perception, la cartographie, la robustesse réseau, la compacité embarquée et la montée en échelle du système.

Chapitre 2

Conception et vue globale du projet

Ce chapitre fixe le cadre architectural d'Auto Fleet. Il établit les choix de séparation fonctionnelle, introduit les sous-systèmes majeurs et précise l'environnement technique ayant servi au développement.

2.1 Architecture technique globale

L'architecture du projet repose sur une logique **distribuée, modulaire et partiellement hétérogène**. Chaque robot conserve des capacités de décision locale, tandis qu'une partie des informations utiles à la mission est remontée, consolidée ou redistribuée à l'échelle de la flotte.

2.1.1 Vue d'ensemble du système

Auto Fleet est conçu comme un système dans lequel plusieurs robots mobiles autonomes opèrent dans un même environnement tout en conservant un degré significatif d'autonomie locale. Chaque unité embarque des capteurs, une puissance de calcul, une chaîne de commande moteur, une alimentation et des capacités de communication lui permettant de traiter son état immédiat sans dépendre en permanence d'un poste central.

L'architecture générale repose sur une séparation entre plusieurs couches fonctionnelles. La première couche correspond à la **perception embarquée** : acquisition de distances, d'images, d'informations inertielles et de données de déplacement. La deuxième couche relève du **traitement local** : filtrage, estimation d'état, détection d'événements, préparation de commandes et mise en forme des informations à transmettre. La troisième couche est celle de la **commande**, chargée de convertir les décisions de haut niveau en signaux effectivement utilisables par les actionneurs.

La flotte n'est pas strictement homogène. Deux robots sont équipés d'un **LiDAR LD06** et participent directement à la **cartographie** de l'environnement. Les deux autres utilisent principalement une **caméra** et leur perception locale pour l'exploration et la progression. Cette dissymétrie fonctionnelle est volontaire : elle permet de mieux répartir les rôles, de limiter le surcoût matériel et de faire émerger une logique de coopération plutôt qu'une simple duplication de plateformes identiques.

Dans cette architecture, la station de supervision ne remplace pas la décision embarquée. Elle fournit une vue consolidée de la mission, centralise certains états, transmet des consignes et rend observables les résultats de la mission. Le système doit donc être lu comme un ensemble dans lequel **autonomie locale** et **supervision globale** coexistent sans s'annuler.

2.1.2 Sous-systèmes embarqués des robots

Les robots embarquent plusieurs sous-systèmes complémentaires dont l'agencement conditionne la cohérence de l'ensemble.

Le premier sous-système est la **chaîne de perception**. Elle comprend, selon les robots, une **caméra**, un **LiDAR**, une **IMU**, des **encodeurs** et des capteurs de proximité. Cette chaîne n'a pas pour seule fonction de détecter des obstacles : elle soutient aussi l'odométrie, la localisation, la navigation et, pour les robots concernés, la production d'une carte exploitable.

Le second sous-système est le **calcul embarqué**. Une carte principale exécute les fonctions de plus haut niveau : lecture des capteurs, préparation des échanges ROS 2, estimation d'état, supervision locale, traitement des consignes et, le cas échéant, exécution des briques de **SLAM**. Cette couche est le centre de décision logiciel du robot.

Le troisième sous-système est la **commande bas niveau**. Il regroupe un microcontrôleur, les divers moteurs et les boucles d'asservissement nécessaires à la production d'un comportement mécanique stable. Ce découplage entre traitement haut niveau et commande temps réel n'est pas un luxe d'implémentation ; il répond à une contrainte de réactivité que la carte principale ne peut garantir seule dans toutes les situations.

Enfin, un dernier sous-système couvre la **communication** et la **gestion énergétique**. Les modules réseau assurent la circulation des états et des consignes, tandis que la distribution électrique doit satisfaire des profils de consommation très différents entre capteurs, calcul embarqué et actionneurs.

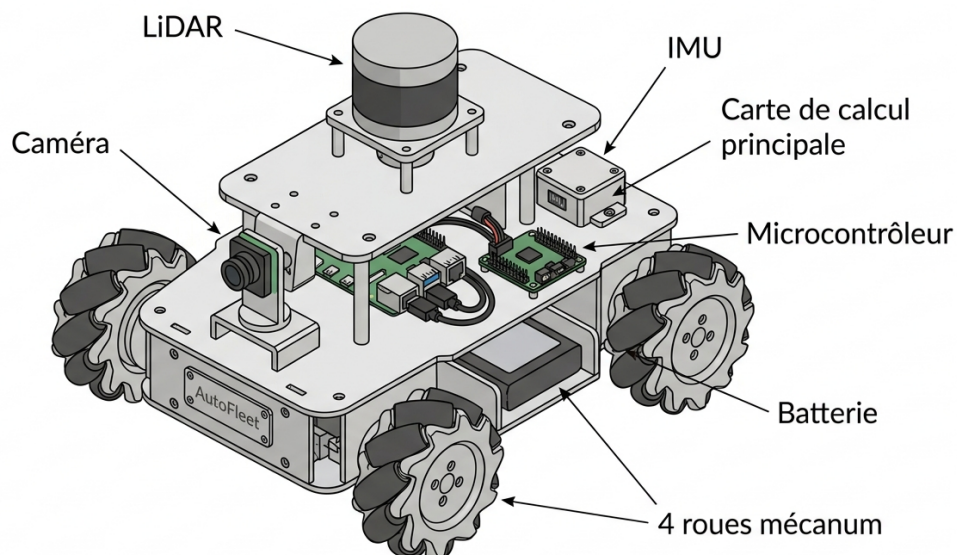


FIGURE 2.1 – Architecture embarquée type d'un robot de la flotte Auto Fleet

2.1.3 Communication et supervision de la flotte

Au-delà du fonctionnement local des robots, Auto Fleet intègre une couche de communication chargée de relier la flotte à une station de supervision. Cette couche transporte des **commandes**, des **états**, de la **télémétrie**, des **événements de mission** et, pour les robots équipés en **LiDAR**, des informations utiles à la **cartographie partagée**.

Les échanges servent d'abord à rendre le système observable. Sans cette remontée d'informations, l'opérateur ne disposerait que d'une perception fragmentaire de la mission. Ils servent ensuite à rendre le système pilotable à un niveau supérieur : lancement d'une mission, arrêt d'un robot, modification d'un mode de fonctionnement ou suivi de l'état global de la flotte.

L'architecture de supervision retenue conserve néanmoins un principe important : la sécurité immédiate et la navigation de proximité restent **embarquées**. Ainsi, une dégradation temporaire du lien réseau ne doit pas suffire à placer un robot dans un état incontrôlé. La supervision complète

l'autonomie ; elle ne s'y substitue pas.

2.2 Environnement de développement du projet

Le développement d'Auto Fleet a nécessité un environnement de travail capable d'articuler **simulation, programmation embarquée, intégration matérielle** et **travail collaboratif**. L'intérêt de cet environnement est moins la diversité des outils que la cohérence de leur usage.

2.2.1 Outils logiciels utilisés

Le développement logiciel s'appuie sur plusieurs ensembles d'outils, chacun correspondant à un niveau distinct du système.

Les fonctions de bas niveau et les interfaces matérielles critiques sont principalement implémentées en **C/C++**, afin de conserver une maîtrise suffisante du matériel, des temps d'exécution et de l'accès direct aux périphériques. Cette couche est particulièrement pertinente pour la commande moteur, la lecture d'encodeurs, les échanges série et les routines proches du temps réel.

Les développements nécessitant davantage de flexibilité reposent sur **Python**. Ce langage a été retenu pour le prototypage rapide, l'outillage interne, certains scripts de test, la manipulation de données capteurs et certaines briques associées à la supervision. Dans ce cadre, la rapidité d'itération et la lisibilité priment sur l'optimisation basse couche.

La colonne vertébrale logicielle du projet demeure **ROS 2**. Ce middleware structure les échanges internes au robot et, selon les cas, entre plusieurs nœuds répartis sur le réseau local. Des outils comme **RViz 2** sont mobilisés pour la visualisation, tandis que les briques de cartographie s'appuient notamment sur **slam_toolbox**. L'objectif n'est pas d'empiler des logiciels, mais de construire une chaîne de traitement intelligible, déboguable et progressive.

2.2.2 Environnement matériel et simulation

Le développement d'Auto Fleet repose sur une approche hybride combinant **plateformes réelles** et **simulation**. Ce choix n'a pas été dicté par confort méthodologique mais par nécessité d'intégration : plusieurs briques pouvaient être validées en simulation alors que le matériel complet n'était pas encore stabilisé.

Le banc d'essai réel est constitué de quatre robots mobiles embarquant une **Raspberry Pi 5**, un microcontrôleur **ESP32**, des moteurs DC encodés, une **caméra NoIR**, une **IMU**, un **LiDAR LD06** pour les robots instrumentés, ainsi qu'une alimentation sur batterie de type **LiPo 3S**. Cette plateforme permet de confronter les hypothèses de conception aux contraintes effectives d'intégration : qualité réelle des mesures, latence des échanges, sensibilité aux vibrations, tenue de l'alimentation et comportement dynamique des châssis.

En parallèle, la simulation a joué un rôle structurant. **Gazebo Sim** a été utilisé pour modéliser les robots et certaines interactions physiques, **RViz 2** pour visualiser les cartes et la télémétrie, et **ROS 2** pour exécuter les nœuds de perception, de navigation et de communication sur une architecture distribuée. La simulation a surtout servi à réduire le coût des itérations logicielles, à éprouver la cohérence des flux de données et à préparer les campagnes d'intégration matérielle.

L'intérêt de cette double approche est concret : elle permet de dissocier, autant que possible, les erreurs de logique algorithmique des défauts d'intégration physique. En pratique, la simulation n'annule pas la nécessité des essais réels ; elle permet simplement d'arriver à ces essais avec des hypothèses déjà filtrées.

2.2.3 Outils de collaboration et de gestion de version

Le travail collectif ne repose pas sur un outil unique, mais sur un petit ensemble d'outils complémentaires.

Le versionnage du code est assuré par **Git**. Son usage ne se limite pas à la conservation d'un historique : il structure les branches de développement, facilite l'isolation des modifications risquées, rend les retours en arrière traçables et permet de distinguer plus proprement les phases d'expérimentation des versions plus stables.

Ce dépôt est associé à une **forge distante**, utilisée pour centraliser les branches de travail, gérer les intégrations successives et conserver une trace des arbitrages techniques. La dimension collaborative ne se réduit donc pas à « partager des fichiers » ; elle implique aussi de rendre visibles les conflits, les fusions, les dépendances et la chronologie des décisions.

En complément, un espace de **partage documentaire** regroupe les schémas, comptes rendus, références techniques et ressources communes du projet. Des canaux de **messaging d'équipe** et, lorsque nécessaire, des outils simples de **suivi de tâches** ont servi à coordonner les tests, à signaler les anomalies et à répartir le travail sans laisser se multiplier des versions concurrentes d'un même sous-système. Le recours à plusieurs outils se justifie ici pleinement : le pluriel du titre renvoie à des besoins distincts, et non à une simple redondance terminologique.

Le **Tableau 2.1** synthétise les principales problématiques techniques traitées dans le projet et les orientations retenues pour y répondre.

Problématique technique	Solution envisagée dans le projet
Déplacement autonome	Utilisation d'une commande moteur bas niveau associée à une logique de navigation embarquée
Perception de l'environnement	Combinaison de plusieurs capteurs : LiDAR , caméra , IMU , encodeurs et capteurs de proximité
Cartographie partagée	Mise en place d'une approche de SLAM sur les robots équipés d'un LiDAR , avec fusion des cartes partielles
Communication de la flotte	Utilisation d'échanges réseau entre robots, backend et station de supervision
Supervision utilisateur	Développement d'une interface graphique permettant de visualiser l'état des robots et l'évolution de la mission

TABLE 2.1 – Principales problématiques techniques du projet Auto Fleet

Chapitre 3

Architecture embarquée des robots autonomes

L'architecture embarquée est ici considérée comme un problème d'intégration complet. Elle engage à la fois la structure mécanique, la distribution d'énergie, l'organisation des calculs, la commande des actionneurs et la capacité du système à rester stable lorsqu'il est effectivement mis en mouvement.

3.1 Contraintes et besoins de l'architecture embarquée

L'identification des contraintes est un préalable nécessaire à tout choix de composants. Une architecture embarquée n'est pas seulement une juxtaposition d'éléments compatibles ; c'est un compromis entre fonctions attendues, limites physiques, exigences de sûreté et capacité réelle d'intégration.

3.1.1 Besoins fonctionnels des robots

Les besoins fonctionnels des robots ne se limitent pas à la locomotion. Chaque plateforme doit fonctionner comme un **nœud autonome de traitement** capable d'assurer simultanément la perception de son environnement, l'estimation de son état, l'exécution de consignes, la remontée d'informations vers la supervision et le maintien d'un comportement sûr en cas de dégradation partielle d'un composant logiciel ou réseau.

La flotte repose sur une **hétérogénéité maîtrisée**. Deux robots, désignés comme **classe A**, embarquent un **LiDAR LD06**, une **IMU** et une base roulante à roues mécanum. Ils sont prioritairement mobilisés pour la **perception géométrique** de l'environnement, la **localisation** et la **cartographie**. Deux autres robots, dits **classe B**, disposent principalement d'une **caméra NoIR**, d'une architecture plus légère et d'un rôle davantage centré sur l'exploration, le déplacement et l'exploitation de données déjà consolidées à l'échelle de la flotte.

Ce découpage répond à une contrainte simple : il n'est ni nécessaire ni souhaitable que chaque robot embarque l'ensemble des capteurs et traitements les plus coûteux. Une flotte efficace ne suppose pas l'uniformité complète ; elle suppose plutôt une répartition des fonctions permettant d'optimiser **coût**, **masse**, **consommation** et **charge de calcul**. Les robots de classe A produisent l'information la plus structurée sur l'environnement, tandis que les robots de classe B peuvent en bénéficier sans supporter la totalité du coût matériel associé.

Sur le plan logiciel, l'architecture s'appuie sur **ROS 2**. Les fonctions sont organisées en nœuds distincts : perception, odométrie, commande, navigation, cartographie, communication et supervision. Cette structuration favorise le découplage des responsabilités et permet une évolution plus propre des sous-systèmes. Les échanges s'appuient sur des topics, des messages horodatés et un modèle de communication cohérent avec les besoins d'une architecture distribuée.

Les robots de classe A exécutent localement une chaîne de **SLAM**, par exemple via `slam_toolbox`. Ils produisent une carte locale à partir des scans **LiDAR** et de l'odométrie enrichie. Cette carte est ensuite transmise au poste de supervision, où peut être réalisée une opération de fusion lorsqu'un recouvrement suffisant existe entre les zones explorées. Les robots de classe B n'exécutent pas cette chaîne complète ; ils peuvent toutefois exploiter les informations consolidées à l'échelle globale.

La communication repose principalement sur le **Wi-Fi** embarqué des **Raspberry Pi**. Grâce au middleware **DDS** de **ROS 2**, la communication interne au système conserve une logique décentralisée. Cette propriété est importante : elle évite qu'un unique point logique ne devienne un goulot d'étranglement pour les échanges de perception et d'état.

Enfin, l'architecture doit rester **évolutive**. Le projet ne vise pas un robot figé, mais une plateforme susceptible d'accueillir des améliorations progressives. Cela impose de préserver des interfaces claires entre les modules et de limiter les dépendances excessivement serrées entre perception, commande, communication et supervision.

3.1.2 Contraintes mécaniques et énergétiques

Les choix techniques sont fortement contraints par le châssis, l'encombrement disponible, la masse embarquée et la distribution de puissance. Même lorsqu'une solution paraît satisfaisante sur le plan fonctionnel, elle peut devenir inexploitable si son intégration physique dégrade la stabilité du robot ou la qualité des mesures.

Chaque plateforme doit accueillir, dans un volume restreint, une carte de calcul, un microcontrôleur, les drivers moteurs, des capteurs, une batterie et l'ensemble du câblage associé. Cette contrainte impose une implantation rigoureuse. Les éléments qui font l'objet de manipulations fréquentes batterie, connecteurs d'alimentation, liaisons série, drivers doivent rester accessibles afin de rendre les tests et la maintenance possibles.

Les robots de classe A sont mécaniquement les plus contraints en raison de la présence du **LiDAR**. Le capteur doit être placé de manière à préserver un champ de vue dégagé à 360 degrés. Une obstruction partielle dans le plan de balayage introduirait des zones aveugles et dégraderait directement la qualité de la carte produite. En outre, le support du capteur doit limiter les vibrations, car une mesure géométriquement précise perd rapidement de sa valeur lorsqu'elle est portée par une structure instable.

Le choix de roues mécanum implique également une géométrie suffisamment rigoureuse. Un mauvais alignement, une dissymétrie d'empattement ou une répartition de masse défavorable peuvent introduire des comportements parasites : glissement accru, rotations non désirées, dérive de trajectoire ou dégradation de l'odométrie.

La contrainte énergétique est tout aussi structurante. La batterie retenue est une **LiPo 3S 5200 mAh**, de tension nominale 11,1 V. Les moteurs et leurs drivers peuvent être alimentés à partir de cette source, tandis qu'un convertisseur **DC-DC** abaisseur fournit le 5 V stabilisé nécessaire à la **Raspberry Pi** et aux périphériques basse tension.

Le **Tableau 3.1** propose une estimation des puissances en jeu pour les deux classes de robots. Ces valeurs restent nécessairement indicatives : la consommation réelle dépend de la charge processeur, du nombre de capteurs actifs, du profil d'accélération et du type de manœuvre demandé.

Composant	Tension	Courant ty- pique	Puissance	A	B
Raspberry Pi 5 8 Go	5 V	3 à 5 A	15 à 25 W	Oui	Oui
ESP32	3,3 V	$\approx 0,24$ A	$\approx 0,8$ W	Oui	Oui
LiDAR LD06	5 V	$\approx 0,5$ A	$\approx 2,5$ W	Oui	Non
IMU 6 axes	3,3 V	$\approx 3,9$ mA	$\approx 0,013$ W	Oui	Non
RaspiCam V3 NoIR	3,3 V	$\approx 0,25$ A	$\approx 0,8$ W	Oui	Oui
4 moteurs DC encodés	7,4 à 12 V	≈ 1 A/moteur	≈ 16 W	Oui	Oui
Drivers moteurs	–	–	≈ 1 W	Oui	Oui
Convertisseur DC-DC	–	–	pertes $\approx 12\%$	Oui	Oui
Puissance totale estimée	–	–	≈ 63 W / 59 W	A	B

TABLE 3.1 – Estimation de la consommation électrique des robots de classe A et B

La capacité énergétique utilisable de la batterie peut être estimée par :

$$E_{utile} = V_{nom} \times C_{bat} \times \eta_{decharge}$$

avec $V_{nom} = 11,1$ V, $C_{bat} = 5,2$ Ah et un rendement de décharge estimé à $\eta_{decharge} = 0,85$. On obtient :

$$E_{utile} = 11,1 \times 5,2 \times 0,85 \approx 49 \text{ Wh}$$

L'autonomie théorique devient alors :

$$t_A = \frac{49}{63} \approx 47 \text{ min} \quad t_B = \frac{49}{59} \approx 50 \text{ min}$$

Ces valeurs constituent une borne haute. En conditions réelles, les transitoires de courant, les rotations, les déplacements latéraux et les charges CPU élevées réduisent sensiblement l'autonomie exploitable. Une durée de fonctionnement utile de l'ordre de **35 à 40 minutes** paraît plus réaliste dans un scénario de mission continue.

3.1.3 Choix d'une architecture embarquée distribuée

Au regard des contraintes précédentes, une architecture embarquée à **deux niveaux de calcul** a été retenue.

Le **niveau supérieur** est assuré par une **Raspberry Pi 5**. Il prend en charge les fonctions de traitement non strictement déterministes : exécution des nœuds **R0S 2**, acquisition capteurs, fusion de données, logique de navigation, communication réseau, supervision locale et, pour les robots de classe A, traitement du **SLAM**. Ce niveau requiert une puissance de calcul significative, mais n'a pas vocation à assurer seul les boucles d'asservissement les plus rapides.

Le **niveau inférieur** est assuré par un **ESP32**. Ce microcontrôleur exécute les tâches les plus proches du temps réel : génération des signaux **PWM**, lecture des encodeurs, boucles **PID** de vitesse et mécanismes de sécurité élémentaires. Cette séparation n'est pas seulement commode ; elle répond à une nécessité pratique. Un système Linux généraliste offre de la souplesse, mais ne garantit pas, à lui seul, une régularité suffisante pour des boucles de commande moteurs exigeantes.

La communication entre les deux niveaux peut s'effectuer via **UART** ou **SPI**. Les messages transportent typiquement des consignes de vitesse, des états moteurs, des mesures d'encodeurs ou des informations d'odométrie. Cette liaison doit rester simple, robuste et suffisamment rapide pour ne pas dégrader la fréquence effective de commande.

L'intérêt de cette architecture réside également dans sa **robustesse fonctionnelle**. En cas de défaillance partielle du traitement haut niveau, le microcontrôleur peut imposer un état sûr, par exemple l'arrêt des moteurs en l'absence de commande valide sur un intervalle défini. La chaîne de commande ne dépend donc pas complètement de l'état instantané de la carte principale.

3.2 Conception matérielle et intégration des robots

Cette section détaille la mise en œuvre matérielle de l’architecture retenue. Elle traite à la fois de la structure physique des plateformes et des flux effectifs traversant les composants embarqués.

3.2.1 Structure mécanique et châssis

Le châssis constitue l’ossature physique de chaque robot. Il ne s’agit pas d’un simple support, mais d’un élément déterminant pour la stabilité mécanique, l’accessibilité des composants et la qualité finale des mesures.

Pour les robots de classe A, l’usage de roues mécanum impose une géométrie rectangulaire et une implantation suffisamment symétrique. La cinématique inverse d’une base à quatre roues peut s’écrire sous la forme :

$$\begin{pmatrix} \omega_{FL} \\ \omega_{FR} \\ \omega_{RL} \\ \omega_{RR} \end{pmatrix} = \frac{1}{r} \begin{pmatrix} 1 & -1 & -(L+W) \\ 1 & 1 & (L+W) \\ 1 & 1 & -(L+W) \\ 1 & -1 & (L+W) \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ \omega_z \end{pmatrix}$$

où r représente le rayon des roues, L le demi-empattement longitudinal, W le demi-empattement latéral, v_x et v_y les vitesses linéaires dans le repère du robot, et ω_z la vitesse de rotation autour de l’axe vertical.

Les notations de cette matrice sont rappelées dans le **Tableau 3.2**. Cette explicitation est utile, car le passage d’un modèle cinématique à une implémentation embarquée est une source fréquente d’ambiguïtés si les conventions géométriques ne sont pas strictement posées.

Symbole	Signification	Unité
r	Rayon d’une roue	m
L	Demi-empattement longitudinal	m
W	Demi-empattement latéral	m
v_x	Vitesse longitudinale du robot	m/s
v_y	Vitesse latérale du robot	m/s
ω_z	Vitesse angulaire autour de l’axe vertical	rad/s
$\omega_{FL}, \omega_{FR}, \omega_{RL}, \omega_{RR}$	Vitesses angulaires des roues avant gauche, avant droite, arrière gauche et arrière droite	rad/s

TABLE 3.2 – Signification des variables de la matrice cinématique des roues mécanum

Ce modèle montre immédiatement qu’une erreur mécanique sur L , W ou sur l’alignement des roues se répercute sur la qualité de la commande et, par conséquent, sur l’odométrie. Dans une plateforme omnidirectionnelle, cette sensibilité est plus marquée que sur une base différentielle classique.

La répartition des masses a également été traitée comme un point critique. La batterie, qui constitue l’un des éléments les plus lourds, est positionnée au plus près du centre du châssis afin de limiter les déséquilibres dynamiques. Cette disposition améliore particulièrement le comportement lors des translations latérales et des rotations rapides.

Pour les robots équipés d’un **LiDAR**, le support mécanique du capteur doit à la fois dégager le plan de balayage et limiter les vibrations. Une fixation insuffisamment rigide dégrade directement la qualité des scans et, par conséquent, celle du **SLAM**. Dans la pratique, plusieurs ajustements de positionnement ont été nécessaires avant d’obtenir une implantation satisfaisante.

3.2.2 Capteurs, cartes embarquées et actionneurs

L'architecture matérielle se compose de plusieurs familles de composants : **capteurs**, **cartes de calcul**, **actionneurs** et **alimentation**.

Les robots de classe A embarquent un **LiDAR LD06**. Ce capteur réalise un balayage 2D à 360°, adapté à la détection d'obstacles et à la production de cartes d'occupation. Sa fréquence de rotation et sa compacité en font un choix cohérent pour une plateforme mobile de taille réduite.

Une **IMU 6 axes** fournit des mesures d'accélération et de vitesse angulaire. Isolément, ce capteur ne suffit pas à assurer une localisation fiable sur longue durée ; en revanche, il constitue une source précieuse pour améliorer l'estimation d'orientation et compenser partiellement certaines dérives d'odométrie sur des horizons temporels courts.

La **RaspiCam V3 NoIR** ajoute une perception visuelle de l'environnement. Dans l'état actuel du projet, cette caméra est principalement exploitée pour l'observation et l'exploration, mais elle constitue également un levier pour des développements ultérieurs en détection visuelle, suivi d'objets ou perception sémantique.

La **Raspberry Pi 5** joue le rôle d'unité de calcul principale. Elle exécute les nœuds **ROS 2**, les traitements capteurs, la logique de communication et, pour les robots de classe A, les briques de cartographie.

L'**ESP32** assure la commande bas niveau. Il lit les encodeurs, applique les régulations **PID**, génère les signaux **PWM** et permet de conserver une commande moteur plus stable que si ces fonctions étaient laissées entièrement à la carte principale.

Chaque robot embarque quatre moteurs DC encodés. Les drivers de puissance, de type pont en H, pilotent les moteurs par paires. Cette organisation est bien adaptée à une base mobile à quatre roues et limite la complexité du câblage sans sacrifier la capacité de commande indépendante de chaque roue.

3.2.3 Intégration des différents sous-systèmes

L'intégration ne peut être réduite au montage physique des composants. Elle englobe aussi les connexions électriques, la hiérarchie des flux de données, les mécanismes d'horodatage et la distribution des responsabilités entre les différentes briques.

L'organisation retenue est centrée sur la **Raspberry Pi**, qui collecte les données capteurs, exécute les traitements de haut niveau et transmet les consignes de déplacement à l'**ESP32**. Celui-ci transforme ensuite ces consignes en commandes moteur et remonte les informations nécessaires à l'odométrie.

Le **Tableau 3.3** résume quelques flux **ROS 2** représentatifs de l'architecture embarquée.

Topic	Type	Émetteur	Récepteur	Fréquence
/robotN/scan	sensor_msgs/LaserScan	LiDAR LD06	slam_toolbox	10 Hz
/robotN/imu/data	sensor_msgs/Imu	IMU	Fusion capteurs	100 Hz
/robotN/odom	nav_msgs/Odometry	ESP32 / Raspberry Pi	SLAM, Nav2	50 Hz
/robotN/map	nav_msgs/OccupancyGrid	slam_toolbox	Supervision	1 Hz
/merged_map	nav_msgs/OccupancyGrid	PC supervision	Robots, RViz 2	1 Hz
/robotN/cmd_vel	geometry_msgs/Twist	Nav2 / supervision	ESP32	20 Hz

TABLE 3.3 – Exemples de flux **ROS 2** utilisés dans l'architecture embarquée

Les robots de classe A produisent des données plus volumineuses et plus structurées, notamment les scans **LiDAR** et les cartes locales. Ces données doivent être correctement horodatées afin de rester exploitables par les algorithmes de localisation et de cartographie. En pratique, l'horodatage côté **Raspberry Pi** au moment de la réception des données issues de l'**ESP32** permet de réduire certaines incohérences temporelles.

C'est également au niveau de l'intégration que les principales difficultés apparaissent : chutes de tension, câblage trop dense, bruit de mesure, vibrations, saturation partielle d'une liaison série ou désynchronisation entre sources. Le bon fonctionnement du système résulte donc autant de la qualité des choix matériels que de la manière dont ces éléments sont reliés et stabilisés dans le temps.

3.3 Validation de l'architecture embarquée

La validation de l'architecture embarquée ne se limite pas à constater qu'un robot se déplace. Elle consiste à vérifier que le système reste cohérent lorsqu'il exécute des trajectoires, alimente plusieurs sous-systèmes simultanément et transmet ses données sans dégradation excessive.

3.3.1 Tests de locomotion et de commande moteur

Les premières validations ont porté sur la chaîne de locomotion. Dans un premier temps, chaque moteur a été testé individuellement afin d'observer la réponse à une consigne et de régler les paramètres d'asservissement. La vitesse réelle estimée à partir des encodeurs a été comparée à la consigne injectée côté **ESP32**. L'objectif n'était pas de produire une identification exhaustive mais de vérifier la stabilité de la réponse et l'absence d'écarts systématiques manifestes.

Dans un second temps, la cinématique globale de la plateforme a été éprouvée sur des mouvements élémentaires : translation avant, translation latérale, déplacement diagonal et rotation sur place. Ces essais permettent de vérifier la cohérence du modèle mécanum, le sens de rotation des roues et la qualité d'exécution des consignes coordonnées.

Sur un déplacement d'environ un mètre, une erreur finale de quelques centimètres reste observable. Cette dérive n'a rien d'anormal dans ce type de base roulante : elle provient des glissements, des imperfections mécaniques et des limites inhérentes au modèle d'odométrie. Les rotations révèlent également une sensibilité notable aux conditions d'adhérence.

Pour les robots de classe A, les essais de locomotion doivent être analysés conjointement avec la stabilité des mesures **LiDAR**. Un déplacement mécaniquement correct mais produisant des scans instables reste problématique, car la qualité de la cartographie dépend directement de la tenue mécanique du capteur.

3.3.2 Problèmes rencontrés lors de l'intégration

Plusieurs difficultés sont apparues au cours de l'intégration.

La première est **mécanique**. L'espace utile sur le châssis s'est révélé plus limité qu'anticipé, en particulier pour les robots intégrant un **LiDAR**. L'implantation simultanée des cartes, des capteurs, des drivers, de la batterie et du câblage impose un arbitrage permanent entre accessibilité, stabilité et compacité.

La deuxième difficulté est **électrique**. Le **Raspberry Pi 5** reste sensible aux chutes de tension, en particulier lors des appels de courant moteurs. Lorsque plusieurs roues accélèrent simultanément, des baisses transitoires du bus d'alimentation peuvent provoquer des sous-tensions, voire des redémarrages partiels de certains éléments.

La troisième difficulté est **logicielle et temporelle**. Une liaison série sous-dimensionnée entre **ESP32** et **Raspberry Pi** peut devenir un facteur limitant dès lors que l'on souhaite transmettre non seulement les consignes, mais aussi l'odométrie, les états moteurs et les informations de diagnostic à fréquence soutenue.

Enfin, la fusion de cartes entre plusieurs robots met en évidence une difficulté d'un autre ordre : la cartographie collaborative dépend fortement de la qualité de l'odométrie, du recouvrement entre zones

explorées et de la stabilité globale de l'architecture. Un problème matériel local peut donc réapparaître, plusieurs couches plus haut, sous la forme d'une incohérence cartographique.

3.3.3 Ajustements et améliorations apportés

Plusieurs corrections ont été introduites à la suite des difficultés précédentes.

Sur le plan électrique, des condensateurs de découplage ont été ajoutés sur le bus moteur afin d'atténuer les chutes de tension transitoires. Côté commande, une limitation du taux de variation des consignes a également été mise en œuvre sur l'**ESP32** pour éviter les appels de courant trop abrupts.

Sur le plan mécanique, la batterie a été recentrée et les supports du **LiDAR** ont été rigidifiés. Ces ajustements, bien que modestes en apparence, ont un effet direct sur la stabilité du robot et sur la qualité des données utilisées par le **SLAM**.

Sur le plan logiciel, la liaison série a été accélérée et le protocole d'échange simplifié. À terme, un encodage binaire compact avec mécanisme élémentaire de contrôle d'intégrité pourrait encore améliorer la fiabilité des flux.

Enfin, l'usage du mode asynchrone de `slam_toolbox` permet de maintenir une charge de calcul plus compatible avec la mobilité réelle du robot. Le choix réalisé ici est explicite : accepter un compromis mesuré sur la finesse de traitement afin de préserver la réactivité globale du système.

3.4 SLAM collaboratif et cartographie partagée

La cartographie collaborative prolonge naturellement l'architecture embarquée. Elle permet de passer d'une perception strictement locale à une représentation plus large de l'environnement, exploitable à l'échelle de la flotte et de la supervision.

3.4.1 Problématique du SLAM multi-agents

Le **SLAM** — *Simultaneous Localization and Mapping* — consiste à construire une carte tout en estimant la position du robot dans cette même carte. Dans Auto Fleet, cette difficulté est déplacée à une échelle supplémentaire : plusieurs robots peuvent produire, en parallèle, des cartes partielles qu'il faut ensuite rendre compatibles.

Deux robots de classe A, équipés d'un **LiDAR LD06**, construisent chacun une carte locale. L'objectif n'est pas seulement de juxtaposer ces cartes, mais de les **fusionner** dès lors qu'un recouvrement suffisant permet d'estimer une transformation cohérente entre leurs référentiels.

Le passage d'un **SLAM mono-robot** à un **SLAM multi-agents** introduit plusieurs contraintes nouvelles : maintien d'une cohérence de référentiel, gestion des délais de communication, qualité du recouvrement entre cartes, sensibilité accrue aux dérives d'odométrie et difficulté d'intégrer des observations partiellement asynchrones.

L'architecture logicielle retenue s'appuie principalement sur `slam_toolbox` pour la construction locale des cartes, sur `multirobot_map_merge` pour la fusion et sur `explore_lite` ou un équivalent pour l'exploration autonome.

3.4.2 Exploration autonome par frontières

L'exploration autonome repose sur le principe des **frontières**, c'est-à-dire des zones situées à l'interface entre l'espace déjà connu comme libre et l'espace encore inconnu. Ce principe a l'avantage de fournir une heuristique simple, directement dérivable de la carte d'occupation.

À partir de la carte, l'algorithme détecte les groupes de cellules inconnues adjacentes à des cellules libres. Chaque frontière F_i , composée de n_i cellules, peut être résumée par un centroïde :

$$cent_i = \frac{1}{n_i} \sum_{j=1}^{n_i} f_{i,j}$$

Un coût simple peut ensuite être défini :

$$cost_i = \alpha \|cent_i - p\|_2 - \beta n_i$$

où p désigne la position actuelle du robot. Le premier terme pénalise les frontières éloignées ; le second valorise celles dont la taille laisse espérer un gain informationnel plus important.

Cette approche est efficace mais imparfaite. Le centroïde d'une frontière n'est pas nécessairement atteignable et peut se trouver trop proche d'un obstacle ou dans une zone localement mal observée. Il doit donc être couplé à un planificateur de trajectoire, par exemple **Nav2**, capable de vérifier la faisabilité du déplacement avant l'envoi effectif d'une consigne.

3.4.3 Fusion des cartes partielles

La fusion des cartes partielles est réalisée sur le poste de supervision. Le nœud `multirobot_map_merge` souscrit aux cartes locales publiées par les robots de classe A et produit une carte globale sur le topic `/merged_map`.

Deux modes de fonctionnement peuvent être distingués. Dans le premier, les poses initiales des robots sont connues. L'algorithme de fusion part alors d'une estimation déjà raisonnable de la transformation entre cartes. Dans le second, ces poses ne sont pas connues et doivent être estimées à partir des zones de recouvrement détectées dans les cartes locales. Ce second cas est plus flexible mais également plus fragile.

Dans le cadre du projet, le mode avec **poses initiales connues** est privilégié, car il réduit la complexité de la fusion et limite l'instabilité lorsque les cartes recouvrent mal les mêmes zones.

Pour une cellule donnée (u, v) , une règle probabiliste simple de fusion peut être écrite :

$$P(M^*(u, v) = occup) = 1 - \prod_{k \in \{1, 2\}} (1 - P(M_k(T_k^{-1}(u, v)) = occup))$$

Cette règle conserve un obstacle détecté par au moins une carte locale tout en maintenant l'état inconnu lorsqu'aucune observation n'est disponible. Elle constitue une solution simple, mais suffisante pour une première cartographie partagée.

3.4.4 Architecture réseau et flux de données

La flotte fonctionne sur un réseau **Wi-Fi local**. Le poste de supervision centralise la visualisation, la fusion des cartes et une partie du suivi de mission. Toutefois, les échanges internes liés à la perception et à la navigation conservent la logique distribuée de **ROS 2** et de **DDS**.

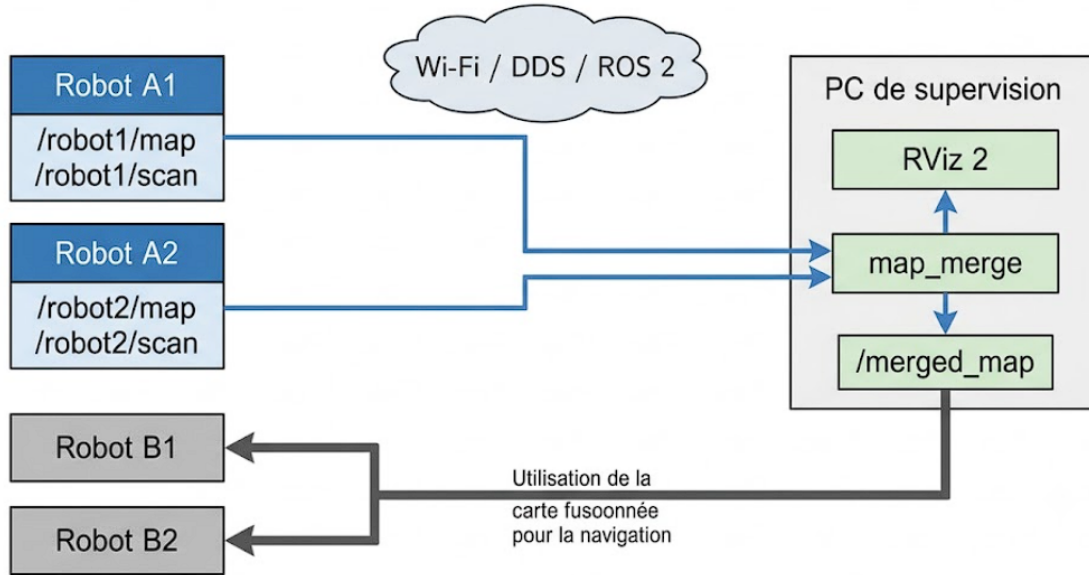


FIGURE 3.1 – Flux de données associés à la cartographie partagée

Les robots de classe A publient leurs cartes locales vers le poste de supervision. Celui-ci calcule une carte globale puis la republie sur `/merged_map`. Les robots de classe B peuvent alors exploiter cette représentation commune sans embarquer eux-mêmes une chaîne complète de cartographie.

Le **Tableau 3.4** donne un ordre de grandeur de la bande passante consommée par quelques flux typiques d'un robot de classe A.

Flux	Taille estimée	Fréquence	Débit brut
LaserScan	≈ 1500 octets	10 Hz	≈ 120 kbps
OccupancyGrid	≈ 250 kB	1 Hz	≈ 2 Mbps
Odometry	≈ 200 octets	50 Hz	≈ 80 kbps
TF	≈ 100 octets	50 Hz	≈ 40 kbps
Total estimé	–	–	≈ 2,3 Mbps

TABLE 3.4 – Estimation de la bande passante utilisée par un robot de classe A

Ce tableau montre que la carte d'occupation constitue le flux le plus coûteux. Une réduction de fréquence, une résolution plus grossière ou un mécanisme de compression peuvent être envisagés lorsque la charge réseau devient trop élevée.

3.4.5 Cohérence globale, limites et validation du SLAM collaboratif

La cohérence globale de la carte dépend fortement du **recouvrement spatial** entre les cartes partielles. Si les robots explorent des zones disjointes, la fusion devient fragile, voire indéterminée. À l'inverse, un recouvrement suffisant améliore considérablement la stabilité de l'alignement.

Les **fermetures de boucle** jouent également un rôle majeur. Une fermeture intra-robot corrige la dérive accumulée le long d'une trajectoire. Une fermeture inter-robots, plus difficile à exploiter, repose sur l'observation d'une même zone par plusieurs unités. Dans l'architecture retenue, ces boucles inter-robots sont principalement prises en compte au moment de la fusion, et non dans une optimisation totalement distribuée d'un graphe unique.

Cette limite est assumée. Une solution plus ambitieuse pourrait reposer sur un échange direct de contraintes de poses, sur de la fusion de scans localisés ou sur une optimisation globale distribuée. Une telle extension serait plus complète, mais aussi plus coûteuse en synchronisation, en bande passante et en complexité logicielle.

La validation du **SLAM collaboratif** ne peut se réduire à l’inspection visuelle d’une carte dans **RViz 2**. Plusieurs métriques quantitatives sont pertinentes, parmi lesquelles l’**ATE** — *Absolute Trajectory Error* — :

$$ATE = \sqrt{\frac{1}{N} \sum_{i=1}^N \left\| \mathbf{x}_i^{SLAM} - \mathbf{x}_i^{ref} \right\|^2}$$

et la **RPE** — *Relative Pose Error* — :

$$RPE(\Delta) = \sqrt{\frac{1}{N - \Delta} \sum_{i=1}^{N-\Delta} \left\| \mathbf{e}_{i,i+\Delta} \right\|^2}$$

À ces métriques de trajectoire peut s’ajouter une comparaison de carte à un plan de référence lorsque celui-ci existe. L’objectif des campagnes de tests finales sera précisément d’objectiver la dérive, la qualité des fermetures de boucle et la cohérence de la fusion sur des parcours connus.

Chapitre 4

Perception de l'environnement par capteurs embarqués

La perception est traitée dans Auto Fleet comme une fonction structurante. Elle ne fournit pas seulement des mesures ; elle conditionne la qualité de la décision locale, la fiabilité de l'odométrie, la robustesse de la cartographie et, indirectement, la pertinence de la supervision.

4.1 Besoins en perception et choix des capteurs

Le choix des capteurs ne découle pas d'une logique d'inventaire. Il répond à des besoins précis de localisation, de sécurité, de compréhension de l'environnement et d'intégration embarquée.

4.1.1 Rôle de la perception dans le projet

La perception constitue le point d'entrée de toute autonomie. Un robot incapable d'observer son environnement ne peut ni éviter un obstacle, ni suivre une trajectoire, ni participer à une mission collective de manière fiable. Dans Auto Fleet, la perception doit donc être lue comme une fonction **opérationnelle** et non comme un simple ajout expérimental.

Au niveau local, chaque robot doit produire une représentation suffisante de son environnement immédiat : présence d'obstacles, orientation, déplacement, variation de cap et éléments visuels utiles à la progression. Cette perception locale supporte la **navigation**, la **sécurité de déplacement** et l'**adaptation comportementale**.

Au niveau global, la perception prend une dimension supplémentaire. Deux robots de la flotte sont chargés de fournir une représentation géométrique structurée de la zone explorée afin d'alimenter une **cartographie partagée**. Les données qu'ils produisent n'ont donc pas seulement une utilité embarquée ; elles contribuent à une représentation collective de la mission.

La chaîne de perception intervient également dans la communication vers la station de supervision. Selon leur nature, les données sont diffusées via **ROS 2** à l'intérieur de la pile robotique, puis relayées vers le backend et la supervision via **MQTT** lorsqu'elles doivent être exposées à l'opérateur.

4.1.2 Justification du choix des capteurs

L'architecture de perception retenue repose sur cinq familles de capteurs.

Le **LiDAR LD06** est embarqué sur deux robots. Il fournit une mesure de distance dans un plan horizontal complet, ce qui le rend particulièrement adapté à la détection d'obstacles, au recalage géométrique et à la cartographie 2D. Sa compacité et son intégration correcte dans l'écosystème **ROS**

2 en font un choix cohérent pour une plateforme légère.

La **caméra NoIR 12 Mpx** équipe l'ensemble des robots. Elle apporte une perception visuelle riche, utile à l'exploration, à l'observation de la scène et à des développements futurs en vision par ordinateur. Pour les robots dépourvus de **LiDAR**, elle constitue une source centrale de perception environnementale.

L'**IMU** fournit les accélérations linéaires et vitesses angulaires. Elle joue un rôle important dans l'estimation d'orientation et dans la stabilisation des estimations de pose sur des horizons temporels courts.

Les **encodeurs** intégrés aux motoréducteurs mesurent la rotation des roues. Ils sont indispensables au calcul de l'odométrie, notamment sur une base omnidirectionnelle à roues mécanum où chaque roue participe, selon une géométrie précise, à la translation et à la rotation globales.

Enfin, des **capteurs ultrasoniques** complètent la chaîne de perception à très courte portée. Leur rôle n'est pas de remplacer le **LiDAR**, mais de renforcer la sécurité de proximité dans des zones où le plan de balayage laser ou l'interprétation visuelle pourraient laisser subsister une ambiguïté.

Le **Tableau 4.1** récapitule ces familles de capteurs.

Capteur	Modèle	Rôle principal	Robots concernés
LiDAR	LD06	Cartographie, détection d'obstacles à 360°	2 robots
Caméra	NoIR 12 Mpx	Perception visuelle, exploration	Tous
IMU	Intégrée	Orientation, stabilisation d'odométrie	Tous
Encodeurs	37Y3530-43.8	Odométrie, retour de rotation des roues	Tous
Ultrasons	MaxBotix LV-MaxSonar-EZ3	Détection d'obstacles à très courte distance	Tous

TABLE 4.1 – Capteurs retenus dans l'architecture de perception d'Auto Fleet

4.1.3 Limites et complémentarité des capteurs

Aucun des capteurs retenus ne suffit, isolément, à couvrir l'ensemble des besoins du projet.

Le **LiDAR LD06** est très efficace pour la géométrie locale, mais il reste limité à un plan horizontal. Les obstacles hors de ce plan — objets suspendus, variations brusques de niveau, obstacles fins ou très particuliers sur le plan optique — peuvent être mal détectés.

La **caméra** restitue une scène riche et texturée, mais elle ne fournit pas directement une distance exploitable sans traitement supplémentaire. Elle demeure aussi sensible aux conditions d'éclairage et aux contraintes de calcul que ses algorithmes d'exploitation imposent.

L'**IMU** dérive lorsqu'elle est intégrée sur longue durée. Les **encodeurs** sont, quant à eux, affectés par les glissements de roues, phénomène notable sur des bases mécanum. Les **ultrasons**, enfin, restent utiles à courte distance mais offrent une résolution directionnelle limitée.

L'intérêt de la chaîne de perception réside précisément dans la **complémentarité** de ces sources. Le **LiDAR** apporte la structure géométrique, l'**IMU** corrige certaines dérives d'orientation, les **encodeurs** fournissent une mesure continue du déplacement relatif, la **caméra** enrichit la compréhension de la scène et les **ultrasons** renforcent la sécurité à proximité immédiate.

4.2 Traitement et exploitation des données capteurs

Cette section décrit la transformation des mesures brutes en informations utilisables pour la navigation, la cartographie et la supervision.

4.2.1 Acquisition et filtrage des données

L'acquisition des données capteurs est organisée sous **ROS 2**. Chaque source est associée à un nœud chargé de lire les données sur l'interface matérielle correspondante et de les publier dans un format cohérent avec la suite de la chaîne.

Le **LiDAR** publie des messages de type **LaserScan**, la **caméra** des messages **Image**, l'**IMU** des messages **Imu**, et les données issues des **encodeurs**, lues côté **ESP32**, sont remontées puis converties en états articulaires ou directement en messages d'odométrie.

Les données brutes ne sont pas utilisées sans prétraitement. Pour le **LiDAR**, un filtrage par seuil élimine les mesures hors plage utile, tandis qu'un filtrage de cohérence temporelle permet d'atténuer certaines valeurs aberrantes dues à des réflexions parasites ou à des perturbations transitoires.

Pour l'**IMU**, un filtre complémentaire ou une fusion plus avancée permet de stabiliser l'estimation d'orientation en combinant les apports du gyroscope et de l'accéléromètre. Ce point est particulièrement important dans une plateforme soumise à des vibrations et à des accélérations non négligeables.

Les mesures **ultrasoniques** sont lissées par moyenne glissante sur un petit nombre d'acquisitions successives. Ce lissage réduit le bruit sans compromettre la réactivité de la détection de proximité.

Les données de la **caméra**, enfin, peuvent être prétraitées par correction de distorsion et ajustement des paramètres visuels lorsque les conditions d'éclairage le nécessitent. Cet effort reste justifié dès qu'une exploitation visuelle plus élaborée est envisagée.

4.2.2 Fusion des informations issues des capteurs

La fusion capteurs vise à produire une estimation de l'état du robot plus robuste qu'une simple juxtaposition de mesures indépendantes.

Dans Auto Fleet, une première couche de fusion combine les données des **encodeurs** et de l'**IMU** afin de produire une odométrie enrichie. Le calcul basé uniquement sur les encodeurs souffre des glissements et des imperfections mécaniques ; l'**IMU** apporte une contrainte complémentaire, particulièrement utile pour l'orientation.

Cette fusion est mise en œuvre dans **ROS 2** à l'aide du package **robot_localization**, qui implémente notamment un **EKF** — *Extended Kalman Filter*. Le filtre exploite les incertitudes des différentes sources et publie une estimation de pose plus stable sur **/odom**.

Pour les robots équipés d'un **LiDAR**, une seconde correction intervient au niveau du **SLAM**. Le recalage des scans sur la carte en construction compense une partie de la dérive accumulée par l'odométrie. Les robots non équipés de **LiDAR** reposent, en l'état, sur la fusion encodeurs-**IMU** comme source principale de localisation relative.

La synchronisation temporelle est ici déterminante. Un désalignement entre la pose estimée et le scan correspondant peut dégrader le recalage et, par effet indirect, la qualité de la carte produite. L'horodatage précis des messages constitue donc une contrainte technique à part entière.

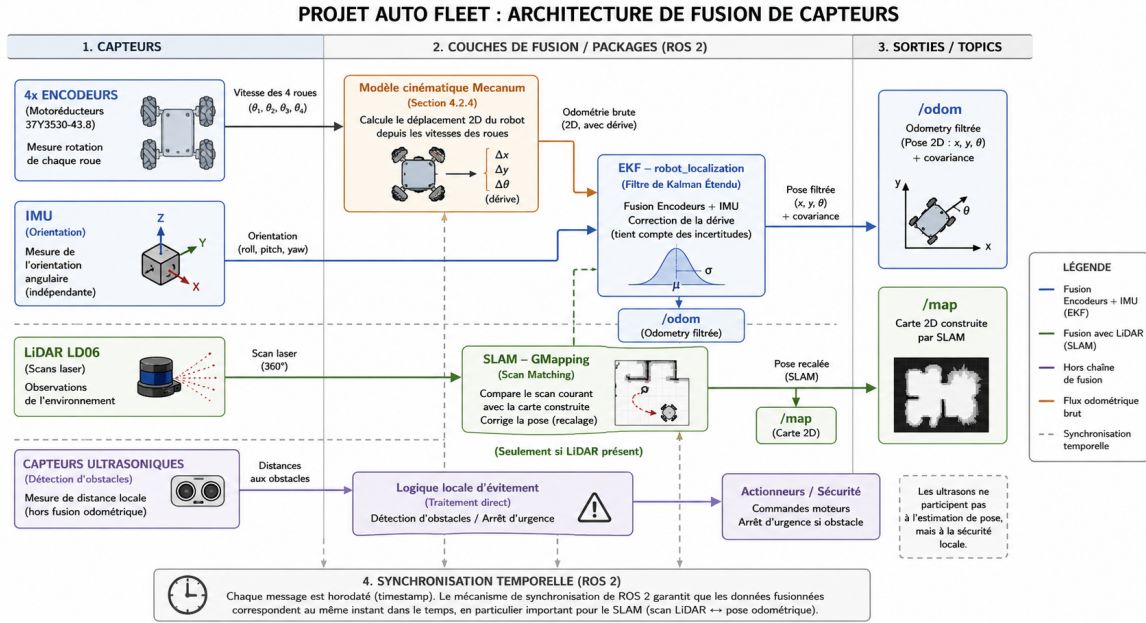


FIGURE 4.1 – Chaîne de perception et d'estimation d'état du projet Auto Fleet

4.2.3 Cartographie partagée par SLAM

Le **SLAM** est l'un des pivots techniques du projet. Il permet de construire progressivement une carte tout en estimant la pose du robot dans cette même carte, sans a priori exhaustif sur l'environnement.

Dans Auto Fleet, cette fonction est assurée par `slam_toolbox`. L'algorithme exploite les scans **LiDAR** et l'odométrie enrichie pour corriger la trajectoire estimée, détecter des fermetures de boucle et mettre à jour la carte d'occupation.

Les robots de classe A produisent chacun une carte locale. Ces cartes sont ensuite fusionnées pour fournir une représentation plus large de la zone explorée. Cette carte partagée a plusieurs usages : soutien à la supervision, lecture de l'avancement de mission, et base potentielle de navigation pour d'autres unités.

Le **SLAM** ne doit pas être lu uniquement comme un résultat cartographique. Il joue aussi un rôle de correction de pose. Une carte cohérente traduit généralement une chaîne perception-odométrie-recalage suffisamment stable ; une carte dégradée signale au contraire une faiblesse structurelle quelque part dans cette chaîne.

4.2.4 Estimation d'état et prise de décision locale

L'estimation d'état regroupe la pose du robot, sa vitesse, son orientation et une représentation simplifiée de son environnement immédiat. Cette estimation résulte de toute la chaîne de perception et constitue la base de la décision locale.

Sur les robots à roues mécanum, le modèle cinématique permet de passer des vitesses de roue au déplacement global du robot et réciproquement. Ce modèle alimente le calcul d'odométrie, mais il doit être interprété à la lumière des limites mécaniques réelles : une cinématique théoriquement correcte peut être dégradée par le glissement, les tolérances de montage ou les irrégularités du sol.

À partir de l'état estimé, le robot prend des décisions locales de navigation. La détection d'un obstacle proche, signalée par le **LiDAR** ou les capteurs **ultrasoniques**, conduit à une adaptation de trajectoire ou à un arrêt temporaire. Cette logique doit rester embarquée afin de préserver la réactivité du système, indépendamment de la latence de supervision.

Les informations d'état significatives sont enfin remontées vers la chaîne de supervision par l'intermédiaire du backend et du broker MQTT. L'opérateur accède ainsi à une représentation synthétique de la situation de chaque robot sans intervenir dans les boucles de sécurité locales.

4.3 Évaluation des performances de perception

L'évaluation de la perception a pour fonction d'identifier ce qui est déjà exploitable et ce qui demeure insuffisamment robuste pour être considéré comme stabilisé.

4.3.1 Détection d'obstacles et estimation de distance

Les premiers essais ont porté sur la capacité des robots à détecter des obstacles et à en estimer la distance dans des conditions contrôlées.

Le **LiDAR LD06** s'est montré particulièrement pertinent pour les obstacles situés dans son plan de balayage. Il fournit des mesures de distance compatibles avec les besoins de navigation et de cartographie, avec une résolution angulaire suffisante pour produire des contours exploitables par les algorithmes de recalage.

Les capteurs **ultrasoniques** se sont révélés utiles à très courte distance, en particulier dans des configurations où le **LiDAR** ne couvre pas exactement la zone la plus proche du châssis. Leur apport n'est pas de remplacer la perception géométrique fine, mais de compléter la sécurité locale.

Pour les robots sans **LiDAR**, la détection rapide d'obstacles repose encore principalement sur les ultrasons, la caméra étant davantage mobilisée pour l'observation et l'exploration. Cette situation souligne un axe d'amélioration clair : l'exploitation plus poussée de la perception visuelle.

4.3.2 Robustesse des mesures en environnement réel

Les conditions réelles dégradent toujours, dans une certaine mesure, la qualité de la perception.

Les vibrations mécaniques générées par les roues mécanum ajoutent du bruit aux mesures **IMU** et peuvent perturber le **LiDAR** si le montage n'est pas suffisamment rigide. La qualité du sol joue également un rôle important, car elle conditionne l'adhérence et, par conséquent, la dérive des estimations fondées sur les encodeurs.

Les conditions d'éclairage influencent fortement la perception par **caméra**. La version **NoIR** améliore la sensibilité en faible luminosité, mais ne supprime pas l'impact des contre-jours, des zones d'ombre ou des variations rapides d'exposition.

La présence d'objets mobiles complique enfin l'exploitation géométrique de l'environnement. Un **SLAM** conçu pour un monde essentiellement statique peut être déstabilisé par des cibles en mouvement si aucun filtrage adapté n'est mis en œuvre. Ces difficultés ne remettent pas en cause la chaîne de perception ; elles définissent plutôt les limites de robustesse actuellement observables.

4.3.3 Limites actuelles et pistes d'amélioration

Plusieurs limites apparaissent déjà avec suffisamment de netteté pour orienter les développements futurs.

La première concerne la **couverture spatiale** du **LiDAR** 2D, limitée à un plan horizontal. Les obstacles hors plan ou les variations de niveau restent partiellement hors champ. Une amélioration possible consisterait à enrichir la perception verticale, soit par vision, soit par capteurs additionnels.

La deuxième limite touche à la **dérive odométrique**, particulièrement sensible sur une base mécanum lorsque les déplacements latéraux deviennent fréquents. Même avec fusion capteurs, une erreur

persistante peut s'accumuler. Des recalages externes plus riches ou des marqueurs visuels pourraient, à terme, réduire cette dépendance.

La troisième limite est liée à l'**exploitation de la caméra**. Son potentiel reste largement supérieur à son usage actuel. Des développements de vision par ordinateur permettraient d'ajouter une dimension sémantique à la perception, absente d'une chaîne purement géométrique.

Enfin, la **calibration extrinsèque** entre capteurs et la **fusion des cartes multi-robots** constituent deux points où un formalisme plus poussé améliorerait sensiblement la qualité des résultats. Ces améliorations ne supposent pas nécessairement un changement de matériel ; elles demandent surtout une montée en maturité de la chaîne de traitement.

Chapitre 5

Communication entre robots et supervision (V2X)

La communication constitue l'une des composantes structurantes du projet **Auto Fleet**. Elle ne sert pas uniquement à transmettre quelques ordres depuis une interface graphique ; elle rend possible l'observation continue de la flotte, la coordination entre plusieurs robots, l'intégration de flux visuels et la qualification de l'état réseau pendant une mission. Dans un système multi-robots, une architecture logicielle peut être pertinente localement mais devenir inutilisable si les données ne circulent pas avec une latence, une régularité et une lisibilité suffisantes.

Le développement récent du prototype a permis de passer d'une chaîne essentiellement simulée à une chaîne hybride, associant un robot réel basé sur **Raspberry Pi**, une station de supervision sur ordinateur portable, un broker MQTT, un backend applicatif, un worker video et une interface Web. Cette évolution modifie sensiblement la portée du chapitre : il ne s'agit plus seulement de décrire une architecture attendue, mais de documenter une chaîne effectivement mise en œuvre, testée sur un réseau local, et préparée pour l'ajout progressif de capteurs plus riches tels qu'une caméra de profondeur Kinect, un LiDAR et une IMU.

5.1 Enjeux de communication et de supervision de la flotte

L'objectif de la couche V2X est de maintenir un lien exploitable entre les robots, la station de supervision et les différents services logiciels. Cette couche doit rester suffisamment simple pour être robuste dans un contexte de prototype, mais assez structurée pour supporter plusieurs familles de données : commandes, télémétrie, états de services, flux video, alertes, diagnostic réseau et informations capteurs.

5.1.1 Besoins d'échange entre robots et station

Les échanges entre les robots et la station couvrent cinq besoins principaux. Le premier concerne les **commandes**. L'opérateur doit pouvoir sélectionner un robot, changer son mode, émettre une consigne de téléopération, lancer ou interrompre une mission et recevoir un retour confirmant la prise en compte de l'ordre. Ces messages sont peu volumineux, mais leur valeur fonctionnelle est élevée : une commande perdue ou retardée affecte directement la confiance de l'opérateur.

Le deuxième besoin concerne la **télémétrie**. Chaque robot publie périodiquement son état courant : mode, batterie, pose estimée, vitesse, informations réseau et disponibilité du flux video. Dans la version actuelle du prototype, le robot réel **R1** publie ces informations depuis un agent Python exécuté sur la Raspberry Pi. Les données sont consommées par le backend, historisées en mémoire et exposées à l'interface de supervision sous une forme plus directement affichable.

Le troisieme besoin porte sur les **états de service**. La supervision ne doit pas seulement connaitre l'état du robot, mais aussi celui des briques logicielles qui composent la chaine : backend, broker MQTT, worker video, agent embarque, serveur RTSP et processus de streaming. Cette lecture est indispensable pour distinguer une panne de caméra, une interruption réseau, une indisponibilité du broker ou une erreur d'affichage côté navigateur.

Le quatrieme besoin concerne les **flux visuels**. La video n'a pas la meme criticite qu'une commande d'arret, mais elle apporte une comprehension contextuelle essentielle lors d'une demonstration ou d'un diagnostic. Dans l'implémentation rétenu, la Raspberry Pi publie un flux RTSP via MediaMTX, tandis qu'un worker video côté station le convertit en flux MJPEG lisible directement par le navigateur. Cette séparation evite de faire porter au navigateur la complexite de RTSP, tout en conservant une chaine simple a tester.

Enfin, le système doit preparer l'intégration de **capteurs supplementaires**. La caméra Raspberry Pi est déjà exploitée comme flux visuel principal. Les emplacements logiques pour une caméra de profondeur Kinect, un LiDAR et une IMU ont été réservés dans les messages de synthese capteurs. Leur état peut donc être affiche comme **offline**, **degraded** ou **online** sans modifier l'architecture générale au moment ou les pilotes réels seront ajoutés.

Famille de données	Sens principal	Contenu	rôle dans la supervision
Commandes	station vers robot	mode, téléopération, mission, arret	action opérateur et coordination
télémétrie	robot vers station	batterie, pose, état, réseau, horodatage	suivi temps réel de la flotte
accusés de réception	robot vers station	identifiant de commande, statut, latence	verification de prise en compte
états de services	services vers station	disponibilité backend, broker, worker video	diagnostic de la chaine logicielle
Flux video	robot vers worker, puis IHM	RTSP, MJPEG, snapshot	observation visuelle et aide au diagnostic
Synthese capteurs	robot vers station	caméra, Kinect, LiDAR, IMU, fusion	preparation de la perception multi-capteurs
Alertes et événements	robot ou backend vers station	obstacle, deconnexion, anomalie, mission	interprétation de la situation

TABLE 5.1 – Principales familles d'échanges dans la chaine V2X

5.1.2 Contraintes réseau et priorités fonctionnelles

Le réseau Wi-Fi utilise pendant les essais ne peut pas être considéré comme une ressource parfaitement stable. Les adresses IP changent lorsque le robot et l'ordinateur basculent d'un environnement réseau à un autre ; les règles de pare-feu peuvent bloquer certains ports entrants ; la qualité radio varie selon la position du robot et la charge du réseau. Ces contraintes ont guidé la mise en place d'une architecture capable de se reconfigurer sans modifier manuellement tout le code.

La première contrainte est la **latence**. Elle est critique pour la téléopération et importante pour la lisibilité de la supervision. Une latence ponctuellement élevée ne rend pas nécessairement le système inutilisable, mais elle dégrade la sensation de contrôle et peut masquer un changement d'état important.

La deuxième contrainte est le **jitter**. Dans une interface de supervision, une valeur moyenne faible ne suffit pas : si les messages arrivent de manière irrégulière, les courbes deviennent difficiles à interpréter et les indicateurs de disponibilité peuvent osciller. Le prototype mesure donc la latence, le jitter, le RTT de commande et l'âge des derniers messages reçus.

La troisième contrainte est la **bande passante**. Les messages MQTT restent légers, alors que la vidéo et certaines données de perception peuvent être beaucoup plus coûteuses. Le worker video

introduit donc des profils de qualité permettant d'adapter la resolution et la compression au contexte d'utilisation. Les débits affichés dans l'IHM sont exprimés en **KB/s** et **MB/s**, avec une base de calcul de 1024, afin d'éviter les confusions entre bits par seconde et octets par seconde.

Indicateur	Interprétation	Décision associée
Age du heartbeat	temps ecoule depuis le dernier signe de vie du robot	declarer un robot en ligne ou hors ligne
Latence télémétrie	delai estime de remontee des données	evaluer la fraicheur des informations affichees
RTT commande	delai entre envoi d'un ordre et retour associe	qualifier la réactivité opérateur
Jitter	variabilite temporelle des messages	détecter une supervision instable
débit en KB/s ou MB/s	charge effective ou estimee des flux	choisir un profil video adapte
disponibilité RTSP/MJPEG	état du flux caméra et du proxy navigateur	séparer panne video et panne de télémétrie
RSSI ou qualité radio	qualité indicative du lien Wi-Fi	anticiper une deconnexion ou une degradation

TABLE 5.2 – Indicateurs réseau utilises pour qualifier la supervision

5.1.3 Place de la supervision dans l'architecture globale

La supervision se situe au-dessus de l'autonomie embarquée. Elle ne remplace ni les boucles de commande locales, ni les réactions de sécurité de bas niveau. Son rôle consiste a rendre l'état global comprehensible, a fournir une interface d'action, a centraliser les diagnostics et a conserver une lecture cohérente de la mission.

Dans le prototype, cette couche est organisee autour d'un backend applicatif. L'IHM ne dialogue pas directement avec chaque robot ; elle interroge le backend via une API HTTP. Le backend dialogue avec le broker MQTT, maintient l'état courant de la flotte, consolide les messages recus et expose des structures homogenes au navigateur. Ce choix reduit le couplage entre l'interface et les details de transport.

La supervision prend egalement en charge la mediation avec la video. Les navigateurs ne savent pas lire nativement un flux RTSP dans des conditions simples et portables. Le worker video devient donc un intermédiaire spécialisé : il ouvre le flux RTSP du robot, capture les images, applique un profil de qualité et expose un flux MJPEG ainsi qu'une URL de snapshot.

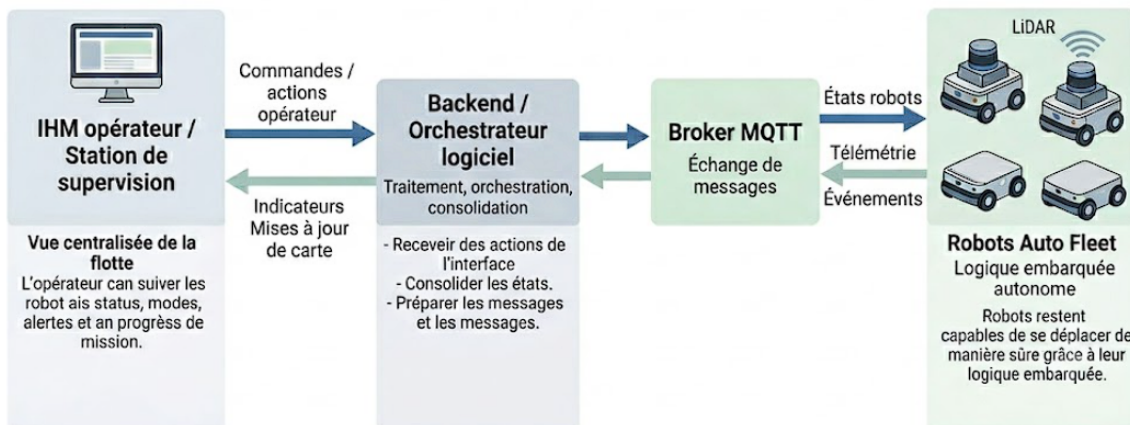


FIGURE 5.1 – Position de la supervision dans l'architecture globale

5.2 Architecture V2X mise en oeuvre

La chaîne développée repose sur une séparation claire entre la station de supervision, les services locaux et le robot embarqué. Cette séparation s'est avérée importante pendant les essais, car elle a permis d'isoler les causes d'erreur : pare-feu Windows, changement d'adresse IP, flux caméra indisponible, broker MQTT non joignable, ou encore processus RTSP arrêté sur la Raspberry Pi.

5.2.1 Decoupage logiciel de la station

La station de supervision exécute plusieurs services complémentaires. Le broker MQTT local écoute sur une adresse accessible depuis le réseau local. Le backend FastAPI consomme les messages, publie les commandes et fournit une API REST à l'IHM. Le worker video gère les flux caméra et publie lui aussi son état via MQTT. L'interface Web, servie par un serveur HTTP simple en développement, affiche l'ensemble des données et permet d'agir sur les robots.

Composant	Port typique	Responsabilité
Frontend Web	3000	interface de supervision, téléopération, video wall, diagnostic réseau
Backend API	8201	API REST, aggregation des états, commandes, protocol status
Broker MQTT local	3890	transport asynchrone entre robots et services de supervision
Worker video	8401	ingestion RTSP, conversion MJPEG, snapshots, profils de qualité
Perception worker	variable	consommation des images ou snapshots pour traitements de perception

TABLE 5.3 – Services exécutés sur la station de supervision

Le choix de MQTT pour la supervision complète l'usage de ROS 2 prévu à l'intérieur des robots. ROS 2 reste pertinent pour les flux techniques embarqués, la navigation et la perception locale. MQTT, de son côté, offre une interface plus simple pour la station, la publication d'états et la dissociation entre producteurs et consommateurs de messages. Cette complémentarité évite de faire porter à l'IHM la complexité complète du middleware robotique.

5.2.2 Organisation des topics et des messages

Les messages sont organisés autour d'un préfixe logique `fleet/v1`. Cette convention permet de distinguer les familles de flux et de préparer la coexistence de plusieurs robots. Les messages de télémétrie, de heartbeat, d'accusé de réception, d'état video, de perception et de synthèse capteurs peuvent ainsi être traités de manière séparée par le backend, tout en conservant un format général cohérent.

Topic logique	Producteur principal	Utilisation
<code>fleet/v1/telemetry/R1</code>	agent Raspberry Pi	état courant du robot et données réseau
<code>fleet/v1/heartbeat/R1</code>	agent Raspberry Pi	détection de présence et disponibilité
<code>fleet/v1/command/R1</code>	backend	commandes de mode, mission ou téléopération
<code>fleet/v1/ack/R1</code>	agent robot	retour sur l'exécution ou la réception des commandes
<code>fleet/v1/video_status/R1</code>	worker video	disponibilité RTSP/MJPEG, résolution, FPS, débit estimé
<code>fleet/v1/sensor/R1</code>	agent robot	état caméra, Kinect, LiDAR, IMU et fusion capteurs
<code>fleet/v1/heartbeat/backend</code>	backend	surveillance des services de supervision

TABLE 5.4 – Exemples de topics utilisés dans la chaîne V2X

Cette structuration a deux avantages. D'une part, elle rend l'ajout d'un robot supplémentaire peu intrusif : il suffit de publier les memes familles de messages avec un autre identifiant. D'autre part, elle permet au backend de conserver une logique d'état par robot, plutôt qu'une simple collection de messages bruts. L'IHM peut donc afficher une vision consolidée, incluant le dernier état connu, l'âge du heartbeat, la disponibilité video et les capteurs réservés.

5.2.3 Agent Raspberry Pi et services persistants

Sur le robot réel, l'agent embarqué est exécuté sur une **Raspberry Pi**. Il publie la télémétrie, annonce son heartbeat, expose l'URL RTSP détectée et remonte une synthèse capteurs. La caméra est publiée séparément via un serveur **MediaMTX** et un script de streaming utilisant la caméra Raspberry Pi.

Afin de rendre la chaîne exploitable après un redémarrage, les services suivants ont été installés et activés via **systemd** :

- `autofleet-mediatrix.service`, charge de démarrer le serveur RTSP local ;
- `autofleet-stream.service`, charge de publier la caméra Raspberry Pi vers **MediaMTX** ;
- `autofleet-agent.service`, charge de publier la télémétrie, le heartbeat et l'état capteurs vers MQTT.

Cette persistance est importante pour un prototype mobile. Elle évite de dépendre d'une session terminal ouverte sur la Raspberry Pi et permet de redémarrer le robot sans relancer manuellement toute la chaîne. Les scripts `install_raspi_autofleet_services.sh` et `restart_raspi_autofleet_services.sh` documentent cette installation et facilitent sa reproduction.

5.2.4 Adaptation aux changements de réseau

Une difficulté concrète rencontrée pendant l'intégration a été le changement d'environnement réseau. Lorsque le robot et l'ordinateur basculent d'un Wi-Fi à un autre, leurs adresses IP peuvent changer. Le prototype intègre donc deux mécanismes complémentaires.

Le premier mécanisme est l'**auto-découverte**. Des scripts peuvent scanner le sous-réseau local, tester les ports connus et détecter les endpoints RTSP ou MQTT disponibles. Cette capacité réduit

le nombre d'hypothèses fixes dans le code et accélère la remise en service après un changement de routeur.

Le second mécanisme est la configuration explicite du chemin MQTT dans le démarrage réel. Lorsque le pare-feu Windows bloquait les connexions entrantes vers le broker, un tunnel SSH inverse a été utilisé temporairement. Une fois ce blocage supprimé, la configuration normale a été basculée vers un chemin direct sur le réseau local : la Raspberry Pi se connecte directement au broker de l'ordinateur. Cette solution supprime une couche de redirection, simplifie le diagnostic et réduit la latence potentielle des messages de télémétrie et de commande.

Le script `start_real_r1_stack.ps1` automatise ce démarrage : lancement du broker, du backend, du worker video, du frontend, configuration de l'adresse MQTT côté Raspberry Pi, puis redémarrage des services embarqués. Le mot de passe de la Raspberry Pi n'est pas inscrit en dur dans le script ; il est fourni par paramètre ou par variable d'environnement.

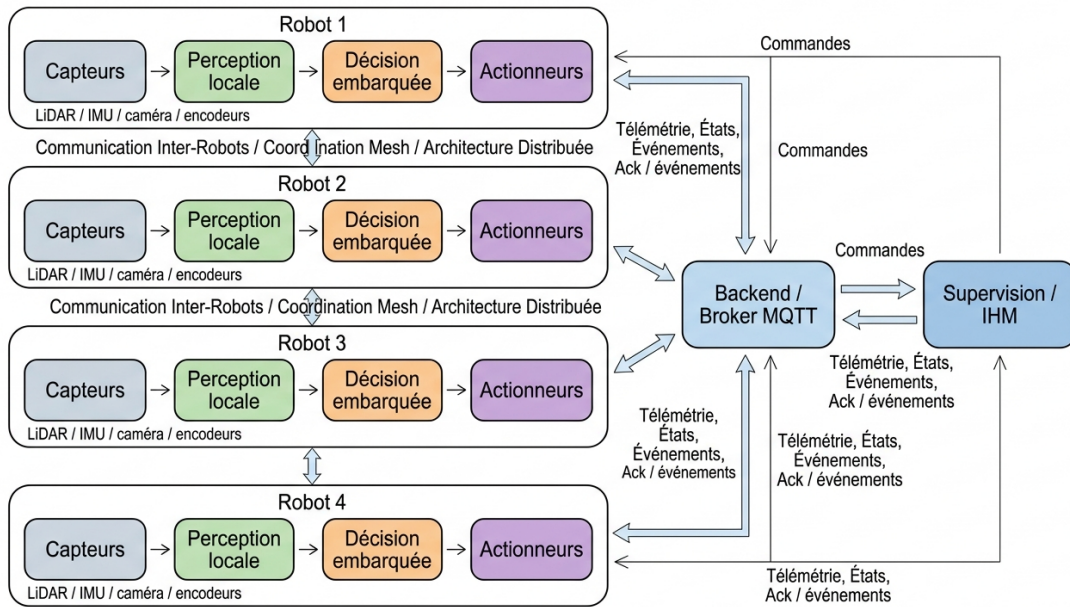


FIGURE 5.2 – Architecture logicielle complétée de la communication V2X

5.3 Supervision Web et pilotage opérateur

L'IHM a progressivement évolué d'une interface de test vers un poste de supervision plus complet. Elle regroupe aujourd'hui le suivi de flotte, la téléopération, les commandes de mission, les courbes réseau, la video wall, les alertes, le statut protocolaire et un laboratoire de diagnostic.

5.3.1 Vue flotte et état consolidé

La vue principale présente les robots connus sous forme de tableau. Pour chaque robot, elle affiche l'identifiant, l'état en ligne, le mode, la batterie, l'âge du dernier message, la latence, le jitter, le débit, le RTT de commande et la disponibilité RTSP. Ces informations proviennent de familles de messages distinctes, mais sont réunies par le backend en une seule représentation.

Cette consolidation évite à l'opérateur de raisonner topic par topic. Il peut savoir rapidement si un robot est joignable, si sa caméra est disponible, si sa télémétrie est récente et si le lien réseau se dégrade. La supervision devient ainsi un outil de décision, et non une simple console de logs.

5.3.2 téléopération et commandes de mission

L'interface expose un mode de téléopération par clavier ainsi que des formulaires de commande. Les consignes sont envoyées au backend, qui les publie sur le topic de commande du robot cible. Le système distingue les commandes ponctuelles et les consignes régulières de téléopération. Cette distinction est importante, car une commande de mission peut être historisée comme événement, tandis qu'une consigne de téléopération est plus fréquente et doit rester réactive.

La présence d'accusés de réception permet de mesurer le RTT de commande. Cette mesure donne une indication plus proche de l'expérience opérateur que la simple latence de télémétrie. Elle permet aussi de détecter un robot encore présent mais lent à réagir aux ordres.

5.3.3 Video wall et profils de qualité

La video wall affiche les flux disponibles sous une forme directement lisible dans le navigateur. Le worker video récupère le flux RTSP du robot, génère un flux MJPEG et publie périodiquement l'état du flux : résolution effective, fréquence d'image, débit estimé, URL de snapshot et statut.

Quatre profils de qualité ont été introduits :

Profil	Largeur max.	qualité JPEG	Intervalle MJ-PEG	Usage visé
Ultra Low Latency	360 px	54	40 ms	priorité à la réactivité
Balanced	640 px	68	30 ms	compromis par défaut
Quality	960 px	76	35 ms	meilleure lisibilité visuelle
High Quality	1280 px	82	45 ms	observation détaillée si le réseau suit

TABLE 5.5 – Profils de qualité disponibles pour la video wall

Un problème observé pendant le développement venait de l'IHM : le changement de profil était bien accepté par le backend et le worker video, mais l'interface affichait parfois un message d'échec parce qu'une fonction de rafraîchissement globale manquait après l'application du profil. Cette erreur a été corrigée en séparant l'échec réel de l'API de configuration video d'un éventuel échec de rafraîchissement secondaire. La supervision indique ainsi plus fidèlement l'état du système.

5.3.4 Diagnostic réseau et unités de débit

Le diagnostic réseau affiche les courbes de latence, jitter, perte, RTT et débit. Les débits sont présentés en **KB/s** ou **MB/s**. Ce choix a été retenu pour rester cohérent avec la perception utilisateur des volumes de données, en particulier pour la video. Les anciens champs exprimés en **kbps** peuvent encore être lus par compatibilité, mais les nouveaux champs privilégient **throughput_kb_s** pour la télémétrie et **bitrate_kb_s** pour la video.

Le laboratoire de diagnostic permet aussi de simuler une charge video multi-robots. Cette simulation ne remplace pas un test radio complet, mais elle donne une estimation de la capacité soutenable lorsque plusieurs flux sont affichés simultanément. Elle aide à choisir entre un profil de qualité élevé et un profil plus réactif.

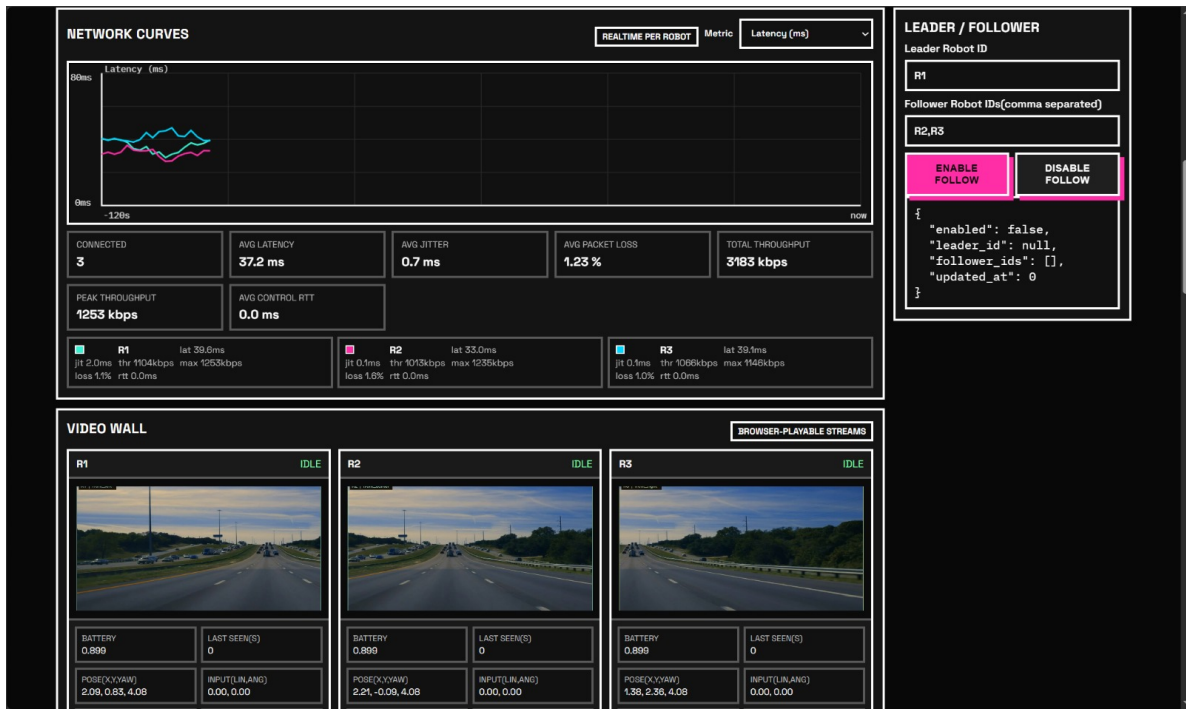


FIGURE 5.3 – Organisation générale de l'IHM de supervision

5.4 intégration du robot réel R1

L'intégration de R1 a constitué une étape importante, car elle a permis de confronter l'architecture logicielle à une vraie Raspberry Pi, un vrai flux caméra et un réseau local domestique. Cette phase a mis en évidence des problèmes que la simulation ne faisait pas apparaître : pare-feu de l'ordinateur, confusion entre sous-réseaux, ports bloqués, processus caméra déjà ouvert et persistance des services au redémarrage.

5.4.1 Chaîne de démarrage rétenu

La chaîne de démarrage finale s'articule autour de deux environnements. Côté ordinateur, le script de lancement démarre les services locaux : broker MQTT, backend, worker video, frontend et worker de perception. Côté Raspberry Pi, les services systemd assurent la disponibilité de MediaMTX, du streaming caméra et de l'agent MQTT.

Le chemin normal de communication utilise un broker MQTT exposé sur le réseau local. Sur le banc d'essai, la Raspberry Pi **R1** se connecte directement à l'adresse de l'ordinateur. Le tunnel SSH inverse, utilisé temporairement pour contourner un pare-feu Windows, n'est plus le mode nominal. Cette évolution simplifie la topologie et rend le comportement plus proche d'une exploitation réelle.

Élément	Implémentation actuelle	État
télémétrie robot	agent Python sur Raspberry Pi, publication MQTT	opérationnel
Flux caméra	caméra Raspberry Pi publiee en RTSP via MediaMTX	opérationnel
Proxy navigateur	worker video RTSP vers MJPEG	opérationnel
démarrage Pi	services systemd persistants	opérationnel
Connexion MQTT	chemin direct Pi vers station	opérationnel
Kinect profondeur	pilotes et emplacements logiques prepares	réservé
LiDAR et IMU	schema de synthese capteurs et états réservés	réservé

TABLE 5.6 – État d’intégration du robot réel R1

5.4.2 Video réelle et reduction de latence

Le flux video réel a été stabilise autour d’une chaine simple : acquisition caméra sur la Raspberry Pi, publication RTSP, ingestion par le worker video, puis exposition MJPEG au navigateur. Les premiers essais ont montre que le probleme principal ne venait pas de l’IHM ni de MQTT, mais de la couche d’acquisition caméra sur la Raspberry Pi. Une fois la caméra stabilisee et les scripts de streaming repris, le flux a pu être observe depuis l’interface.

La reduction de latence ne depend pas d’un seul paramètre. Elle resulte d’un compromis entre resolution, qualité JPEG, fréquence de capture, intervalle MJPEG et charge CPU. Le profil **Ultra Low Latency** réduit fortement la resolution afin de privilegier la réactivité. Les profils **Quality** et **High Quality** augmentent la lisibilité lorsque le réseau et la machine peuvent l’absorber. Cette logique est appliquee a tous les robots affiches par le worker video, ce qui evite d’avoir une politique de qualité differente selon le robot.

5.4.3 Preparation de l’extension capteurs

La supervision intègre déjà une synthese capteurs par robot. Pour R1, la caméra est declaree en ligne lorsque le flux RTSP est disponible. Les entrees Kinect, LiDAR et IMU sont presentes mais marquees hors ligne tant qu’aucun pilote réel ne publie de mesure. Cette approche peut sembler modeste, mais elle evite de modifier le contrat de données au moment de l’ajout de nouveaux capteurs.

La caméra de profondeur Kinect ouvre une perspective interessante : elle permettrait de produire une information de distance plus riche que la simple image couleur. Les composants logiciels nécessaires a son intégration ont été prepares côté Raspberry Pi, notamment les piles **libfreenect**, **libfreenect2** et **OpenNI2**. L’objectif n’est pas d’afficher immediatement un nuage de points dans l’IHM, mais de disposer d’un chemin clair pour publier une distance minimale, un état de flux profondeur ou une image de profondeur reduite.

Le LiDAR et l’IMU suivent la meme logique. Le LiDAR peut alimenter la cartographie et la détection d’obstacles, tandis que l’IMU contribue a l’estimation d’orientation et a la détection de mouvements brusques. Dans tous les cas, les données brutes ne doivent pas être affichees sans mediation : la supervision doit recevoir une synthese exploitable, par exemple un nombre de points, une distance minimale, un taux de rafraichissement, une confiance ou un état de degradation.

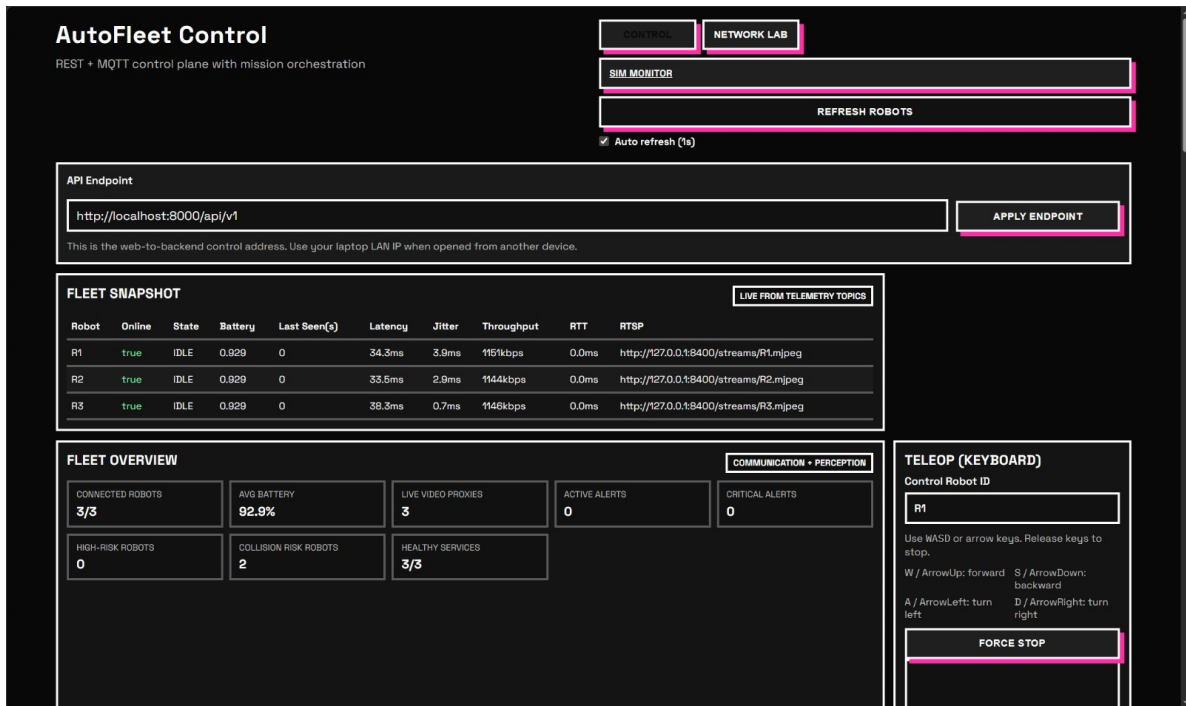


FIGURE 5.4 – Vue de diagnostic réseau et de gestion des flux video

5.5 Validation et analyse des limites

La validation de la chaine V2X repose sur une combinaison de tests unitaires, d'observations en conditions réelles et de mesures visibles dans l'IHM. Le passage au robot réel a permis de mieux distinguer les composants fonctionnels de ceux encore prospectifs.

5.5.1 Resultats obtenus

Les resultats les plus importants sont les suivants. Le backend recoit la télémétrie de R1 via MQTT et l'affiche dans la flotte. Le worker video détecte le flux RTSP, le convertit en MJPEG et publie son état. L'IHM affiche les indicateurs réseau en KB/s ou MB/s, les courbes temps réel, la disponibilité caméra et les profils de qualité. Les services sur Raspberry Pi sont persistants et redemarrent après reboot. Enfin, la connexion MQTT fonctionne en chemin direct sur le réseau local lorsque le pare-feu de la station l'autorise.

Ces resultats valident le rôle du backend comme couche de consolidation. Les informations visibles dans l'IHM ne proviennent pas d'une seule source : elles combinent télémétrie robot, heartbeat, état video, état service et synthese capteurs. La station peut ainsi presenter une lecture cohérente meme lorsque certains flux sont absents.

5.5.2 Limites observees

Plusieurs limites demeurent. La premiere concerne la dependance au réseau local. Un changement de sous-réseau, une adresse IP stale ou une regle de pare-feu peuvent suffire a interrompre la télémétrie. Les mécanismes de decouverte et la configuration directe reduisent ce risque, mais ne l'annulent pas totalement.

La deuxieme limite concerne la video. La chaine RTSP vers MJPEG est simple et compatible navigateur, mais elle n'est pas aussi optimale qu'un protocole video temps réel spécialisé. Pour le prototype, ce compromis est acceptable, car il privilegie la reproductibilite et le diagnostic. Pour un

usage plus avance, une solution WebRTC ou un pipeline encode plus finement pourrait reduire la latence.

La troisieme limite concerne la densite d'information. L'IHM affiche de nombreuses informations utiles, mais une flotte plus grande necessiterait une hierarchisation plus forte : filtrés par robot, regroupement par niveau d'alerte, affichage conditionnel des capteurs et priorisation des flux video.

Enfin, les capteurs réservés ne sont pas encore des sources de mesures réelles. Leur presence dans le schema prouve que l'architecture est prête a les accueillir, mais la validation fonctionnelle devra être refaite lorsque la Kinect, le LiDAR et l'IMU publieront effectivement des données.

5.5.3 améliorations possibles

Les prochaines améliorations peuvent être regroupees en quatre axes. Le premier consiste a renforcer la decouverte réseau et la tolerance aux changements d'adresse. Le deuxieme concerne la video : mesure plus précise de la latence de bout en bout, profils adaptatifs selon la charge et eventuelle evaluation de WebRTC. Le troisieme axe porte sur les capteurs : publication de mesures Kinect, intégration LiDAR/IMU et fusion dans un message de synthese plus riche. Le dernier axe concerne l'ergonomie : rendre l'IHM plus selective, mieux differencier les erreurs de transport, de capteur et de service, et conserver un historique d'événements plus exploitable.

Axe	État actuel	Évolution visée
Réseau	direct LAN, decouverte, fallback tunnel possible	selection automatique du meilleur chemin
Video	RTSP vers MJPEG avec profils manuels	adaptation dynamique selon latence et débit
Capteurs	caméra active, Kinect/LiDAR/IMU réservés	publication de mesures réelles et fusion
Supervision	tableaux, video wall, diagnostic réseau	vues filtrées, historique enrichi, priorisation
Persistence	services systemd sur la Raspberry Pi	supervision de redémarrage et logs centralises

TABLE 5.7 – Synthese des limites et perspectives de la chaine V2X

En conclusion, la couche V2X du projet a atteint un niveau de maturite superieur a une simple simulation. Elle relie effectivement une Raspberry Pi, un flux caméra réel, un backend, un broker MQTT, un worker video et une IHM Web. Les limites restantes ne remettent pas en cause l'architecture ; elles definissent plutot les prochaines etapes vers une supervision plus robuste, plus adaptative et plus riche en données capteurs.

Chapitre 6

Rendu final

Ce chapitre est volontairement resserré. Il recevra, dans la version finale du manuscrit, la description stabilisée de ce qui est effectivement démontré : interface finale, scénarios retenus et résultats visibles du point de vue utilisateur.

6.1 Interface utilisateur finale

La description détaillée de l'IHM sera consolidée lorsque l'intégration complète des vues et des retours backend sera figée. Les rubriques suivantes sont conservées afin de structurer cette présentation finale.

6.1.1 Organisation générale de l'IHM

Cette sous-section documentera l'architecture visuelle finale de l'interface : répartition des panneaux, hiérarchie des blocs d'information, zones de commande, espace de retour mission et éventuelles vues cartographiques. L'objectif sera de justifier les choix d'organisation au regard de la **charge cognitive** imposée à l'opérateur.

6.1.2 Interactions entre l'utilisateur et le système

Le texte final précisera les interactions réellement exposées : sélection d'un robot, émission d'ordres, changement de mode, lancement ou interruption de scénario, ainsi que les confirmations ou refus d'exécution remontés par le système. La distinction entre **commande opérateur** et **autonomie embarquée** devra y rester explicite.

6.1.3 Informations affichées et retour du système

Cette sous-section recensera les informations considérées comme essentielles à l'usage : état de connexion, batterie, télémétrie, retour mission, alertes, diagnostics réseau et disponibilité des flux complémentaires. Une approche purement descriptive sera évitée au profit d'une lecture fonctionnelle de l'affichage.

6.2 Tests utilisateur et certification

Cette section rassemblera les éléments de validation du prototype du point de vue de l'usage. Elle devra distinguer ce qui relève d'un comportement démontré, d'un comportement partiellement stabilisé ou d'un comportement encore expérimental.

6.2.1 Scénarios de test retenus

Les scénarios retenus pour l'évaluation finale préciseront les conditions de démonstration, les tâches confiées à l'opérateur et les fonctions effectivement mobilisées. Il s'agira de décrire des cas d'usage lisibles, et non une accumulation de manipulations élémentaires.

6.2.2 Résultats observés du point de vue utilisateur

Cette sous-section présentera les résultats visibles à l'échelle de l'interface : cohérence des états affichés, réactivité perçue des commandes, pertinence des retours et lisibilité générale de la supervision.

6.2.3 Autres tests et limites actuelles

Les autres campagnes de validation — robustesse, stabilité ou contraintes réseau — seront synthétisées ici de manière à relier l'expérience utilisateur aux limites techniques encore présentes dans le prototype.

Chapitre 7

Gestion de projet

La gestion de projet a joué un rôle important dans le déroulement du projet **Auto Fleet**. Le projet ne se limitait pas à un développement logiciel classique : il combinait de la mécanique, de l'électronique, de la communication, de la supervision, de la perception et de l'intégration physique. Cette diversité a rendu impossible une organisation strictement linéaire. Nous avons donc adopté une méthode de travail itérative, proche d'une démarche agile, afin de pouvoir avancer malgré les imprévus matériels et les ajustements techniques. Nous avons réparti le travail autour de grands axes : la construction du robot, la partie vision et perception, la cartographie, l'IHM, ainsi que la communication entre les robots et entre les robots et l'interface de supervision. Cette organisation nous a permis de travailler en parallèle tout en gardant des points de synchronisation réguliers.

7.1 Méthode de gestion

La méthode de gestion retenue repose sur une logique **agile et itérative**. Dans le cadre du projet de synthèse, une méthode classique comme le cycle en cascade ou le cycle en V n'était pas adaptée, car plusieurs choix techniques dépendaient des essais réels, de la disponibilité du matériel et des problèmes rencontrés pendant l'intégration.

Nous avons donc avancé par versions successives. Chaque période de travail avait pour objectif de produire un résultat observable, même partiel : un robot capable de bouger, une première communication PC-robot, une simulation exploitable, une lecture de capteurs, une interface de supervision ou encore une démonstration intermédiaire. Cette approche a permis de corriger progressivement les problèmes au lieu d'attendre la fin du projet pour réaliser l'intégration complète.

Des réunions avec les tuteurs ont été réalisées environ toutes les deux semaines. Ces réunions servaient à présenter l'état d'avancement, identifier les blocages, valider les choix techniques et redéfinir les priorités lorsque cela était nécessaire.

7.1.1 Cycle de vie agile utilisé

Le cycle de vie suivi pendant le projet peut être résumé en plusieurs itérations courtes. Chaque itération commençait par un objectif technique, puis se poursuivait par une phase de développement, de test, de correction et de revue avec les tuteurs.

Étape de l'itération	Objectif	Application dans le projet
Développement	Produire une première version exploitable.	Développement du code, préparation des fichiers 3D, tests de montage, création de scripts ou configuration de la communication.
Test intermédiaire	Vérifier si la solution fonctionne dans un cas simple.	Tests sur simulation, essais de montage, vérification des déplacements, échanges réseau simples, affichage dans l'IHM.
Correction	Modifier la solution en fonction des problèmes observés.	Réimpression de supports moteurs, correction du code, modification de la structure du châssis, adaptation du planning.
Revue	Présenter l'avancement et décider de la suite.	Réunions avec les tuteurs, bilan des blocages et choix des priorités pour l'itération suivante.

TABLE 7.1 – Cycle de travail itératif utilisé pendant le projet

Cette méthode a été particulièrement utile pour la partie mécanique. En effet, certains éléments n'étaient pas disponibles au moment prévu ou ne correspondaient pas exactement aux besoins. Au lieu d'arrêter le projet, nous avons déplacé l'effort vers les tâches qui pouvaient continuer : simulation, communication, IHM, préparation des scripts et conception 3D.

7.1.2 Organisation générale des axes de travail

Le projet a été structuré autour de plusieurs axes techniques. Ces axes n'étaient pas indépendants, mais ils permettaient de répartir les tâches et de faire avancer plusieurs parties en parallèle.

Axe de travail	Objectif principal	Exemples de travaux réalisés
Construction du robot	Concevoir et assembler une base mobile exploitable.	Châssis, supports moteurs, intégration des composants, impression 3D, câblage, tests physiques.
Commande et déplacement	Permettre au robot de se déplacer correctement et de manière contrôlée.	Mouvements simples, PID, trajectoires, odométrie, stabilisation des déplacements.
Vision, perception et capteurs	Exploiter les capteurs pour détecter l'environnement proche.	Lecture LiDAR, traitement d'obstacles simples, exploitation des données, tests de perception.
Cartographie et coordination	Préparer la représentation de l'environnement et la coordination des robots.	Carte, flotte simulée, évitement simple, coordination basique, tests d'intégration.
IHM et communication	Permettre le contrôle, l'affichage et la supervision du système.	Communication PC-robot, communication robot-IHM, V2X, heartbeat, timeouts, IHM de démonstration.
Validation et démonstration	Préparer une version présentable et répétable du projet.	Tests globaux, script de lancement, vidéo de secours, slides, documentation technique.

TABLE 7.2 – Structuration du projet Auto Fleet par axes de travail

Cette structuration a permis de limiter les blocages. Par exemple, lorsque le châssis ou les supports moteurs n'étaient pas encore totalement prêts, le développement de l'IHM, de la communication et de la simulation pouvait continuer. À l'inverse, lorsque le robot devenait disponible, les efforts étaient recentrés sur les tests physiques et l'intégration.

7.1.3 Planning prévisionnel et planning réalisé

Le planning du projet s'est étendu principalement de février à mai 2026. Le planning prévisionnel prévoyait une progression régulière : montage du robot, premiers déplacements, lecture des capteurs, communication, pré-intégration, stabilisation puis préparation de la démonstration finale. En pratique, le planning réalisé a dû être adapté à cause des retards et des problèmes liés au matériel, notamment sur le châssis et les supports moteurs.

Période	Prévision initiale	Réalisation effective	Écart constaté
20 février – 6 mars	Montage de base, alimentation, drivers moteurs, premiers mouvements, communication PC-robot.	Travail lancé, mais dépendant de la disponibilité réelle du matériel et des ajustements mécaniques.	Début ralenti par les contraintes matérielles.
7 mars – 30 mars	Odométrie, lecture LiDAR, traitement de données, calibration, premiers tests capteurs.	Avancement logiciel et tests partiels, avec certains essais physiques décalés en raison du montage incomplet.	Validation réelle plus lente que prévu.
31 mars – 7 avril	Pré-intégration technique, flotte simulée, V2X v1, sécurité, affichage et démonstration intermédiaire.	La simulation et les briques logicielles ont permis de présenter une version intermédiaire exploitable.	Objectif maintenu grâce à la simulation.
8 avril – 5 mai	Stabilisation des mouvements, coordination, robustesse réseau et tests globaux.	Recentrage sur la robustesse : heartbeat, timeouts, corrections PID, tests physiques et nettoyage logiciel.	Priorités adaptées aux problèmes observés.
6 mai – 20 mai	Version stable, IHM de démonstration, script de lancement,	Préparation d'une version plus répétable pour la démonstration, avec simplification du lancement.	Objectif global respecté avec adaptation.
21 mai – 12 juin	Slides, documentation	Finalisation des supports, préparation de la soutenance et sauvegarde vidéo.	Phase réalisée comme prévu.

TABLE 7.3 – Comparaison entre le planning prévisionnel et le planning réalisé

Le planning réel n'a donc pas été une simple exécution du planning initial. Il a évolué selon les contraintes rencontrées. Les tâches liées à la mécanique ont parfois pris plus de temps que prévu, tandis que les tâches logicielles ou de simulation ont été avancées pour éviter une période d'attente.

7.1.4 Versions intermédiaires prévues et réalisées

La consigne du projet demandait de présenter les versions intermédiaires prévues et réalisées. Dans notre cas, ces versions correspondaient aux jalons techniques que nous avons utilisés pour suivre l'avancement du projet.

Version	Période / jalon	Objectif prévu	Résultat réalisé
V0	Fin février 2026	Disposer d'une base de robot en cours de montage, avec premiers tests matériels.	Version partielle, ralentie par les problèmes de châssis et de supports moteurs.
V1	6 mars 2026	Présenter une démonstration minimale du robot lors du rendez-vous tuteurs.	Démonstration limitée, avec certaines parties encore en cours de stabilisation.
V2	Mars 2026	Avancer sur la communication PC-robot, les capteurs et les premiers traitements.	Communication et développements logiciels poursuivis, avec des tests parfois réalisés hors robot complet.
V3	7 avril 2026	Pré-soutenance technique : montrer une architecture fonctionnelle à environ 70%.	Version intermédiaire basée sur l'intégration partielle, la simulation et les premiers modules testables.
V4	20 mai 2026	Obtenir une version stable et une démonstration répétable.	Version recentrée sur les fonctions essentielles : mouvement, communication, IHM, sécurité et lancement simplifié.
Version finale	31 mai 2026	Préparer le pack de présentation, les slides, la documentation et une vidéo de secours.	Supports finalisés pour la soutenance, avec une vidéo backup en cas d'instabilité de la démonstration réelle.

TABLE 7.4 – Versions intermédiaires prévues et réalisées pendant le projet

Cette logique de versions a permis de garder une progression mesurable. Même lorsque le système complet n'était pas encore prêt, chaque version intermédiaire apportait une brique exploitable pour la suite.

7.2 Répartition des tâches et outils de gestion

La répartition des tâches a été organisée autour des quatre membres du groupe : **Massyl**, **Jianke**, **Josué** et **Karim**. Chaque membre avait un axe principal, mais la répartition a évolué pendant le projet selon les problèmes rencontrés et les besoins d'intégration.

7.2.1 Répartition initiale des tâches

La répartition initiale avait pour but de couvrir les principales briques du projet. Elle est présentée dans le tableau 7.5.

Membre	Responsabilité principale	Travaux associés
Massyl	Communication, contrôle et IHM	Communication PC-robot, panneau de contrôle, V2X, flotte simulée, durcissement réseau, heartbeat, timeouts, IHM de démonstration et script de lancement.
Jianke	Perception, LiDAR et coordination	Exploitation des données de tests, traitement LiDAR, détection d'obstacles simples, pré-intégration perception, coordination basique et nettoyage algorithmique.
Josué	Montage, capteurs et tests physiques	Montage, alimentation stable, drivers moteurs, encodeurs, odométrie basique, lecture LiDAR brute, mode sécurité, tests physiques globaux et documentation montage/câblage.
Karim	Commande, simulation et trajectoires	Mouvements simples du robot, PID, IMU, calibration, fusion capteurs, stabilisation finale, optimisation des trajectoires, simulation et schémas techniques.
Tous les membres	Validation finale	Intégration globale, tests de démonstration, correction des problèmes, préparation des slides, vidéo de secours et soutenance.

TABLE 7.5 – Répartition des tâches entre les membres du groupe

Cette répartition n'était pas figée. Les parties étaient fortement liées : l'IHM dépendait des données de communication, la cartographie dépendait de la stabilité de l'odométrie, et les tests physiques dépendaient de la qualité du montage mécanique. Il était donc nécessaire de conserver une organisation flexible.

7.2.2 Évolution de la répartition pendant le projet

Au cours du projet, la répartition des tâches a évolué en fonction des blocages et des périodes critiques. Lorsque le robot réel n'était pas encore complètement disponible, nous avons déplacé une partie du travail vers les développements réalisables sans matériel complet : simulation, interface, communication et préparation des scripts.

Les principales évolutions ont été les suivantes :

- les tâches mécaniques ont demandé plus d'itérations que prévu, notamment pour les supports moteurs ;
- les développements logiciels ont parfois été avancés en parallèle pour compenser les retards matériels ;
- les phases d'intégration ont nécessité une collaboration plus forte entre les membres ;
- les dernières semaines ont été consacrées collectivement à la stabilisation, à la démonstration, aux slides et à la vidéo de secours.

Situation rencontrée	Effet sur l'organisation	Réaction adoptée
Matériel non disponible ou incomplet	Certaines tâches de test physique ne pouvaient pas être réalisées immédiatement.	Avancement en simulation, préparation de l'IHM et développement de la communication.
Supports moteurs à ajuster	Le montage du robot a demandé plusieurs essais avant d'être exploitable.	Modification des modèles 3D, passage par Blender et Bambu Studio, puis nouvelles impressions.
Pré-soutenance technique	Besoin de présenter une version intermédiaire cohérente.	Regroupement des efforts sur les briques démontrables : simulation, communication, perception et sécurité.
Démonstration finale	Nécessité d'avoir une version simple à lancer et plus fiable.	Préparation d'un script de lancement, tests filmés et vidéo de secours.

TABLE 7.6 – Évolution de l'organisation pendant le projet

Cette évolution montre que nous avons conservé une logique agile : les responsabilités principales existaient, mais elles pouvaient être adaptées lorsque le projet l'exigeait.

7.2.3 Outils de gestion et de collaboration

Plusieurs outils ont été utilisés pour gérer le projet, partager les fichiers, coordonner les tâches et produire les éléments techniques nécessaires.

Outil	Utilisation dans le projet	Apport
Discord	Communication quotidienne, échanges rapides, partage de captures, messages d'organisation et résolution de problèmes.	Permet de garder un contact régulier entre les membres et de réagir rapidement aux blocages.
GitHub	Stockage et versionnement du code, partage des scripts et suivi des modifications.	Évite la perte de fichiers, facilite le travail collaboratif et permet de conserver un historique du projet.
Planning Gantt	Suivi des tâches, des périodes de travail, des jalons et des responsabilités.	Donne une vue globale de l'avancement et permet d'identifier les périodes critiques.
Bambu Studio	Préparation des impressions 3D, découpage des modèles, réglages d'impression et génération des fichiers pour l'imprimante.	Indispensable pour fabriquer et corriger les supports moteurs et certaines pièces du châssis.
Blender	Modification, vérification ou adaptation de fichiers 3D avant impression.	Permet d'ajuster les dimensions, l'encombrement et la forme des pièces mécaniques.
Outils de simulation	Tests de comportement avant validation sur robot réel.	Permettent de continuer le développement même lorsque le matériel est indisponible ou instable.
Documents partagés	Centralisation des informations, figures, schémas, comptes rendus et supports de présentation.	Facilite la préparation du rapport, des slides et de la soutenance.

TABLE 7.7 – Outils utilisés pour la gestion et la collaboration

Ces outils ont été utilisés de manière complémentaire. Discord servait surtout à la coordination rapide. GitHub permettait de conserver les versions du code. Bambu Studio et Blender étaient essentiels

pour la partie mécanique et impression 3D. Le planning Gantt servait à garder une vision d'ensemble du déroulement du projet.

7.3 Changements rencontrés au cours du projet

Pour la partie libre du chapitre, nous avons choisi le thème suivant : **l'agilité du projet face aux changements pendant le projet et les réactions de l'équipe**. Ce thème correspond bien à notre expérience, car plusieurs changements concrets ont eu lieu pendant le projet, principalement autour du matériel, du châssis et de l'intégration.

7.3.1 Problèmes matériels rencontrés

Les principaux changements sont venus de la partie matérielle. Certains éléments n'étaient pas disponibles au moment prévu, tandis que d'autres ne correspondaient pas exactement aux besoins du projet. Le problème le plus marquant concernait le châssis et les pièces nécessaires à l'intégration des moteurs.

Problème rencontré	Conséquence	Impact sur le projet
Matériel non livré à temps	Certains tests physiques ne pouvaient pas être faits au moment prévu.	Décalage des validations réelles et nécessité d'avancer sur d'autres modules.
Matériel reçu non adapté	Certaines pièces ne correspondaient pas exactement aux contraintes du robot.	Modification du montage et ajustement des choix techniques.
Châssis plus complexe à intégrer que prévu	Difficultés pour placer correctement moteurs, batterie, cartes et câblage.	Allongement de la phase de montage et besoin de revoir l'organisation interne du robot.
Supports moteurs à refaire	Les premières versions n'étaient pas parfaitement adaptées aux dimensions réelles.	Plusieurs versions ont été modélisées, préparées et imprimées avant d'obtenir une solution exploitable.
Tests physiques dépendants du montage	Impossible de valider correctement certains modules tant que le robot n'était pas stable.	Utilisation plus importante de la simulation et report de certaines validations.

TABLE 7.8 – Principaux changements et problèmes matériels rencontrés

Ces problèmes ont montré que la mécanique devait être considérée comme une partie centrale du projet. Un retard sur le châssis ou un support moteur mal adapté pouvait bloquer plusieurs autres parties : les déplacements, l'odométrie, les tests de capteurs et la démonstration.

7.3.2 Itérations sur les pièces 3D

L'impression 3D a joué un rôle important dans la résolution des problèmes mécaniques. Pour les supports moteurs, nous avons dû passer par plusieurs versions avant d'obtenir une pièce utilisable. Chaque version permettait de corriger un défaut observé lors du montage réel.

Le processus suivi était le suivant :

1. vérification ou adaptation du modèle 3D ;
2. modification des dimensions ou des points de fixation avec Blender lorsque cela était nécessaire ;
3. préparation de l'impression dans Bambu Studio ;
4. impression de la pièce ;
5. test de montage sur le châssis ;

6. correction du modèle en fonction du résultat obtenu.

Élément corrigé	Problème observé	Correction apportée
Dimensions du support	La taille ne correspondait pas parfaitement à l'espace disponible sur le châssis.	Modification du modèle 3D et réimpression de la pièce.
Points de fixation	Certains trous ou alignements ne correspondaient pas correctement au moteur ou au châssis.	Ajustement des positions dans le modèle avant nouvelle impression.
Résistance de la pièce	Certaines zones pouvaient être trop fines ou fragiles.	Renforcement de la géométrie et adaptation des paramètres d'impression.
Encombrement	La pièce pouvait gêner d'autres composants ou le câblage.	Simplification de la forme et optimisation de l'espace occupé.

TABLE 7.9 – Corrections réalisées sur les pièces imprimées en 3D

Même si ces itérations ont pris du temps, elles ont permis d'obtenir une solution plus adaptée au robot réel. Cette partie illustre bien la logique agile du projet : tester, corriger, réimprimer, puis tester de nouveau.

7.3.3 Conséquences sur l'organisation

Les changements rencontrés ont modifié l'ordre réel des tâches. Certaines validations ont été reportées, tandis que d'autres ont été avancées. Cela a évité que le projet reste bloqué sur une seule difficulté.

Les conséquences principales ont été les suivantes :

- les tests physiques ont parfois été décalés ;
- les développements logiciels ont été poursuivis en parallèle ;
- la simulation a été utilisée plus tôt et plus souvent ;
- la préparation de l'IHM et des scripts a permis d'avancer sans attendre la stabilisation complète du robot ;
- les dernières semaines ont été davantage concentrées sur la robustesse et la répétabilité de la démonstration.

Cette adaptation a permis de conserver une progression globale malgré les imprévus. Le projet n'a donc pas suivi exactement le planning initial, mais il a gardé une cohérence grâce à la réorganisation régulière des priorités.

7.4 Réactions de l'équipe et solutions apportées

Face aux changements rencontrés, nous avons mis en place plusieurs solutions pour limiter les retards et continuer à produire des résultats. Ces solutions concernent à la fois l'organisation, la simulation, l'impression 3D, la communication et la préparation de la démonstration finale.

7.4.1 Poursuite du travail en parallèle

La première réaction a été de ne pas attendre que tous les problèmes matériels soient résolus pour continuer le projet. Lorsqu'une partie était bloquée, nous avançons sur une autre. Par exemple,

pendant que les supports moteurs étaient en cours de correction, il était possible de travailler sur la communication, l'IHM, les scripts ou la simulation.

Blocage	Travail poursuivi en parallèle	Intérêt
Châssis non stabilisé	Simulation, communication PC-robot, préparation de l'IHM.	Éviter l'arrêt complet du projet.
Supports moteurs à corriger	Modélisation 3D, scripts, tests logiciels et préparation des scénarios.	Utiliser le temps d'attente pour avancer sur les parties indépendantes.
Tests physiques non reproductibles	Vidéo de secours, script de lancement et simplification de la démonstration.	Sécuriser la soutenance et réduire les risques le jour de la présentation.
Intégration complexe	Tests par modules avant regroupement complet.	Identifier plus facilement l'origine des erreurs.

TABLE 7.10 – Réactions mises en place face aux blocages

Cette organisation a permis de maintenir une progression continue. Elle correspond directement au fonctionnement agile attendu dans un projet de synthèse.

7.4.2 Utilisation de la simulation

La simulation a servi de solution intermédiaire lorsque le robot réel n'était pas encore prêt ou pas assez stable. Elle a permis de tester des comportements, de vérifier des échanges de données et de préparer les scénarios avant les essais physiques.

La simulation a été utile pour :

- tester les premiers comportements de déplacement ;
- préparer la logique de flotte ;
- vérifier certaines données utilisées par l'IHM ;
- simuler des positions ou des états de robots ;
- préparer la démonstration avant d'avoir un système réel totalement stable.

Elle n'a pas remplacé les tests réels, mais elle a permis de réduire les risques. Lorsque les tests physiques devenaient possibles, nous pouvions repartir d'une logique déjà partiellement validée.

7.4.3 Adaptations techniques apportées

Plusieurs adaptations techniques ont été réalisées pendant le projet afin de répondre aux problèmes rencontrés.

Adaptation	Objectif	Résultat obtenu
Modification des fichiers 3D	Adapter les pièces aux dimensions réelles du robot.	Supports plus adaptés au châssis et aux moteurs.
Utilisation de Bambu Studio	Préparer correctement les impressions 3D.	Impressions plus propres et meilleure maîtrise des paramètres.
Utilisation de Blender	Modifier ou vérifier les modèles 3D avant impression.	Correction plus rapide des problèmes de forme ou de dimensions.
Durcissement réseau	Rendre la communication plus fiable.	Ajout de mécanismes comme heartbeat, timeouts et gestion automatique des nœuds.
Script de lancement	Simplifier le démarrage de la démonstration.	Réduction des manipulations manuelles et des risques d'erreur.
Vidéo de secours	Prévoir une solution en cas de problème le jour de la soutenance.	Sécurisation de la présentation finale.

TABLE 7.11 – Adaptations techniques réalisées pendant le projet

Ces adaptations ont permis de transformer certains blocages en étapes de correction. Elles montrent que les difficultés rencontrées n'ont pas seulement retardé le projet : elles ont aussi conduit à améliorer certaines parties, notamment la robustesse de la démonstration et la qualité de l'intégration.

7.4.4 Bilan de la gestion agile

Au final, la gestion du projet s'est appuyée sur trois principes principaux : avancer par versions intermédiaires, travailler en parallèle et adapter régulièrement les priorités. Cette méthode était adaptée à la nature du projet, car Auto Fleet combinait plusieurs domaines techniques fortement dépendants les uns des autres.

Le bilan de cette organisation est positif. Même si le planning initial a été modifié par les imprévus matériels, nous avons conservé une progression régulière. Les réunions avec les tuteurs ont permis de garder un cadre de suivi, les outils collaboratifs ont facilité la coordination, et les solutions comme la simulation ou l'impression 3D ont permis de continuer à avancer malgré les blocages.

Cette expérience montre que l'agilité n'a pas seulement été une méthode théorique dans notre projet. Elle a été nécessaire pour réagir aux difficultés concrètes : matériel incomplet, pièces à refaire, intégration plus longue que prévu, besoin de versions intermédiaires et préparation d'une démonstration finale fiable.

Chapitre 8

Conclusion et perspectives

8.1 Conclusion du projet

Cette section synthétisera les objectifs atteints, les résultats effectivement obtenus et les limites structurelles demeurant ouvertes au terme du projet. Elle devra distinguer clairement les éléments **démontrés**, **partiellement stabilisés** ou encore **prospectifs**.

8.2 Perspectives d'amélioration du projet

Les perspectives porteront notamment sur l'amélioration de la perception, la robustesse du **SLAM collaboratif**, la fiabilisation réseau, l'enrichissement de l'IHM et la montée en maturité de l'architecture embarquée.

Bibliographie

- [1] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [2] B. Siciliano, O. Khatib, *Springer Handbook of Robotics*, 2^e édition, Springer, 2016.
- [3] OASIS, *MQTT Version 5.0*, Standard, 2019.
- [4] Open Robotics, *ROS 2 Documentation*, documentation technique.
- [5] S. Macenski et al., *slam_toolbox*, documentation et dépôt logiciel.

Annexe A

Annexes

Cette partie regroupera les schémas détaillés, extraits de configuration, résultats complémentaires de tests et éléments de support non intégrés au corps principal du rapport.